Build It

A Complete Step-by-Step Guide to the Final Project

Web Application with React · 40% of the course grade · Deadline 13 November 2026

A reference build of Shelf, a book discovery app on the Open Library API, with every step tied to the rubric, every code file tested before it was written down, and a report template at the end.

Contents overleaf · Work through it in order · Commit at every checkpoint

Build It: A Complete Step-by-Step Guide to the Final Project

Course: Web Application with React · **Weight:** 40% · **Deadline:** 13 November 2026

Reference app in this guide: Shelf — a book discovery app built on the Open Library API.

> **How to use this guide.** Every step tells you what to do, why it matters, and which rubric line it earns. Code blocks are complete files: create the file, paste the block, save. Each part ends with a commit, so your git history tells the story of the build (that is worth 6 of the 40 points on its own). Every piece of code in this guide was compiled and tested before it was written down — search, detail pages, empty states, error states, the lot.
>
> **Read it once end to end before you start typing.** Ten minutes of reading will save you a day of "why is my page blank".

Contents

Part	What you do	Time	Page
0	Understand what is actually being graded	20 min	3
1	Choose a topic and a real API	45 min	4
2	Plan three screens on paper	45 min	5
3	Git, GitHub and the Vite project	45 min	7
4	Bootstrap and a custom theme	40 min	10
5	Routing skeleton and the shared layout	60 min	12
6	The API layer (`fetch`, `async/await`, errors)	90 min	17
7	One reusable data-fetching hook	45 min	20
8	Reusable UI components	90 min	24
9	Screen 1 — Home	90 min	26
10	Screen 2 — Explore (search, sort, paginate)	120 min	29
11	Screen 3 — Book detail	90 min	33
12	Screen 4 — Author page	60 min	36
13	Screen 5 — Reading list (saved state)	90 min	38
14	Styling, responsiveness and accessibility	120 min	42
15	Quality assurance before you ship	60 min	50
16	Deploy to GitHub Pages	60 min	53
17	The README	30 min	55
18	The report (400–600 words)	120 min	56
19	The submission ZIP	20 min	58
20	Final rubric self-check	15 min	59

That is roughly 20 hours of focused work. Spread over six evenings it is comfortable; crammed into one night it is not. Start today.

Part 0 — Understand what is being graded

Most students lose points on things that take ten minutes to fix: a README with no run instructions, a deployed site that is blank because of one config line, or three screens that all do the same search. Read the rubric as a checklist, not as prose.

Rubric line	Points	What an examiner actually looks for
Core structure	0–8	Vite + React, components with clear responsibilities, **3+ screens with different purposes**, a router
API integration	0–8	A real REST/JSON API that fits the topic, `fetch` + `async/await`, loading state, error state, empty state
UI and UX	0–8	Familiar patterns, consistent navigation, works at 320 px and at 1440 px, keyboard usable, alt text, labels
Libraries	0–5	At least one library for **styling** and one for **functionality**, used where they actually help
Version control + deployment	0–6	Real history (not one "final" commit), README that works for a stranger, live public URL that works
Report	0–5	All four sections, concrete examples from your code, honest AI statement

Two hard rules people trip over:

1. **"Three different screens" means three different jobs.** Three URLs that all show a search box and a grid is one screen copied three times. The reference app in this guide has five screens: browse (Home), search (Explore), inspect one item (Book), inspect a person (Author), and manage a personal collection (Reading list).
2. **Weather apps are banned.** So is anything that is just a form that posts to an API and prints the raw JSON.

The 3-screen sanity test. Write one sentence per screen: "On this screen the user does ___ so that ___." If two sentences are interchangeable, you have one screen twice.

Part 1 — Choose a topic and a real API

1.1 What makes an API usable for this project

The assignment says "a real REST API returning JSON data". Beyond that, pick one that survives being called from GitHub Pages:

Requirement	Why it matters	Quick test
`https://` endpoint	Pages is served over HTTPS; mixed content is blocked	The URL starts with `https`
No key, or a key you can safely expose	A static site has no server to hide secrets in	Read the docs: "no authentication required"
CORS enabled	The browser blocks cross-origin reads unless the API says otherwise	See 1.3 below
Stable, documented shape	You will write code against its field names	Docs have example responses
Enough data for 3 screens	You need lists, details and something to filter	It returns more than one field

1.2 Where to find one

- https://github.com/public-apis/public-apis — searchable list, filter by "no auth".
- https://github.com/public-apis/public-apis#index — the index by category.
- If a course topic interests you (music, football, space, books, art, food), look for the official API of an organisation in that field first. They are usually better documented than hobby APIs.

1.3 The CORS check (do this before you commit to an API)

Open the API URL directly in your browser. You will see JSON — that proves nothing, because typing a URL is not a cross-origin request. Instead:

1. Open https://example.com in a tab.
2. Open DevTools → Console.
3. Paste and run:

```
fetch('https://openlibrary.org/search.json?q=dune&limit=1')
  .then((r) => r.json())
  .then((d) => console.log('CORS OK', d.numFound))
  .catch((e) => console.error('CORS BLOCKED', e))
```

If you see `CORS OK`, the API will work from your deployed site. If you see a red CORS error, that API cannot be called from a browser at all — pick another one. Do not "fix" this with a proxy on day five; you will not have time.

1.4 The reference topic: Open Library

Open Library (https://openlibrary.org/developers/api) is an Internet Archive project that catalogues tens of millions of books. It is a good fit because:

- no API key, no sign-up, HTTPS, CORS open;
- three genuinely different data types — **works** (a book), **authors**, and **subjects** — which gives you three different screens almost for free;
- cover images served from `covers.openlibrary.org`, so your UI looks like something;
- it fails in interesting ways (missing descriptions, missing authors, 404 covers), which means your error and empty states are real, not decorative.

The endpoints used in this guide:

Purpose	Endpoint	Notes
Free-text search	`GET /search.json?q=dune&limit=24&page=1`	Returns `numFound` + `docs[]`
Search by author	`GET /search.json?q=author_key:OL79034A&sort=rating`	`author_key` is a field query
Browse a subject	`GET /search.json?q=subject:fantasy`	`subject:` is a field query
Sort results	`...&sort=rating`, `new`, `old`	Anything else falls back to relevance
One work	`GET /works/OL893414W.json`	Title, description, subjects, covers
One author	`GET /authors/OL79034A.json`	Name, bio, dates, photo id
Cover image	`https://covers.openlibrary.org/b/id/11481354-M.jpg`	`-S`, `-M`, `-L` sizes

Two quirks you must design around, and which make excellent report material:

1. **The rating is not on the work endpoint.** `/works/{id}.json` has no rating or first publish year; those fields live in the search index. So the detail screen fetches both endpoints at once with `Promise.all` and merges them.
2. `description` **and** `bio` **are sometimes a string, sometimes an object** like `{ "type": "/type/text", "value": "…" }`, and they contain Windows line endings. Your normalisation layer has to handle both.

1.5 If you want a different topic

You can keep every architectural decision in this guide and swap the API. Pick the mapping that matches your taste:

Topic	API	Screen 1	Screen 2	Screen 3	Extra
Books (this guide)	Open Library	Featured shelves	Search + filters	Book + author	Reading list
Space	`https://api.spacexdata.com/v4`	Next launch + countdown	Launch history w/ filters	Rocket detail	Favourites
Art	`https://api.artic.edu/api/v1`	Highlighted works	Search + department filter	Artwork detail	Personal gallery
Anime	`https://api.jikan.moe/v4`	Seasonal picks	Search + genre	Anime + characters	Watchlist
Classic texts	`https://gutendex.com/books`	Popular books	Search + language	Book + similar	Reading queue
Football	`https://api.football-data.org/v4` (key)	League table	Fixtures by matchday	Team detail	Followed teams

Whatever you choose, **write the three screen sentences before you write code**. If you cannot fill the "extra" column with something a user would want, the topic is too thin.

> **Note on keys.** If an API needs a key, put it in a `.env` file and use `import.meta.env.VITE_API_KEY`. Vite inlines `VITE_*` variables into the built JavaScript, so the key is public — only do this for free, rate-limited, revocable keys, and say so in the README. Never commit `.env`. This is a good paragraph for the report.

Part 2 — Plan three screens on paper

This is the step people skip and then regret. Fifteen minutes with a pen saves hours of refactoring.

2.1 The screen map

Write this table for your own topic. For Shelf:

Screen	Route	The user's job	Data source
Home	`/`	Get inspired and jump in	4 subject searches in parallel
Explore	`/explore?q=…&sort=…&page=…`	Find a specific book	`/search.json`
Book	`/book/:id`	Decide whether to read it	`/works/{id}` + search
Author	`/author/:id`	See everything by one writer	`/authors/{id}` + search
Reading list	`/reading-list`	Keep what they chose	`localStorage` only

Note what makes these different: one is editorial, one is a query tool, two are detail views of different entities (a book and a person are not the same screen), and one is a personal collection with no API call at all. That last one is worth points: it shows you did not just wrap an API.

2.2 Sketch the layout

Draw boxes. Rough is correct. For the Home screen:

```
+-------------------------------------------------------------+
| Shelf            Home   Explore   Reading list (2)  [Search] |  sticky navbar
+-------------------------------------------------------------+
|  POWERED BY OPEN LIBRARY                                    |
|  Find the book you                                          |
|  did not know you wanted.                                   |
|  [ □ Title, author or subject ..... ] [ Search ]            |
|  In the mood for  [fantasy] [sci-fi] [mystery] [romance] ...  |
+-------------------------------------------------------------+
|  Browse the shelves                          Explore all ->  |
|  Fantasy                                      See all        |
|  [cover][cover][cover][cover][cover][cover]  -> scrolls       |
|  Science fiction                                            |
|  [cover][cover][cover][cover][cover][cover]                   |
+-------------------------------------------------------------+
```

Explore, on desktop and on a phone:

```
DESKTOP                                    PHONE (<= 767px)
+----------------------------------------+ +------------------+
| Explore the catalogue                  | | Shelf      [menu]|
| [□ search...........] [sort v] [o] [ ]| | Explore          |
+----------------------------------------+ | [□ search....]   |
| 48,280 results for "dune"  [chips]     | | [sort v] [o] [ ] |
+----------------------------------------+ | 48,280 results   |
| [cover] [cover] [cover] [cover]        | | [cover] [cover]  |
|  Dune    Dune   Dune    Dune           | |  Dune    Dune    |
| [cover] [cover] [cover] [cover]        | | [cover] [cover]  |
+----------------------------------------+ +------------------+
|            < 1 >                      |
```

The important insight from the second sketch: a 4-column grid becomes 2 columns, and the toolbar stacks vertically. You do not need a separate mobile design — you need a plan for how each block collapses.

2.3 The component tree

```
App
├── SiteNavbar            (nav + search dialog, reads the saved count)
├── Routes
│   ├── /            → HomePage      → ShelfRow → CoverImage
│   ├── /explore     → ExplorePage   → BookCard (grid) | ErrorState | EmptyState | skeleton
│   ├── /book/:id    → BookPage      → CoverImage, Stat, AuthorCard, Tabs
│   ├── /author/:id  → AuthorPage    → BookCard (grid), sort buttons
│   ├── /reading-list→ ReadingListPage → Table | BookCard, Modal
│   └── *            → NotFound
└── Footer
```

Rules of thumb that produce a good score here:

- A component that is used on two screens moves to `components/` (`BookCard`, `CoverImage`, `ErrorState`, `Skeleton`).
- A component used by one screen stays in `pages/` as a small local component (`AuthorCard` lives inside `BookPage.jsx`, `Stat` too). Do not over-abstract.

- No component both fetches data and renders a whole page. Fetching goes in the page or in a hook; rendering goes in small components.

Checkpoint. You now have: a topic sentence, a table of three-plus screens with different jobs, two sketches, and a component tree. Commit nothing yet — git comes next.

Part 3 — Git, GitHub and the Vite project

3.1 Install the tools

You need **Node.js 20.19+ or 22.12+** (Vite 8 requires it) and npm.

```
node -v     # v22.x or v20.19+ — if older, install from https://nodejs.org (LTS)
npm -v      # 10.x is fine
git --version
```

Also install the **React Developer Tools** browser extension. You will use it to check that components re-render when they should, and it makes debugging state much faster.

3.2 Create the repository

Do this **before** scaffolding, so `create-vite` does not complain about a non-empty folder, and so your very first commit is already a real project.

1. On GitHub: **New repository** → name it `shelf` (or your app's name) → **Public** → do not add a README, `.gitignore` or licence (you will create those locally).
2. Then, in a terminal:

```
mkdir shelf && cd shelf
git init -b main
```

> Use `main`, not `master`. GitHub Pages and most CI assume `main`.

3.3 Scaffold with Vite

```
npm create vite@latest . -- --template react
```

Answer the prompts; when it asks to install and start, say no for now. You get:

```
shelf/
├── index.html
├── package.json
├── vite.config.js
├── public/
└── src/
    ├── App.jsx
    ├── App.css
    ├── index.css
    ├── main.jsx
    └── assets/
```

Notes on the template as of Vite 8 / React 19:

- It ships **oxlint** instead of ESLint. That is fine, and faster; keep it (see 3.6).

- `src/App.css` and the demo counter in `App.jsx` are placeholder content. Delete them when you build the real screens; do not delete `index.css`.
- TypeScript? The assignment does not require it. This guide uses plain JavaScript, which keeps the report honest about "code you can explain". If you already know TypeScript, use the `react-ts` template and add types to the API layer — it is a real bonus.

3.4 Install the libraries

The rubric asks for third-party libraries for **styling** and **functionality**. This guide uses four, each doing a job nothing else does:

```
npm install bootstrap react-bootstrap react-bootstrap-icons react-router-dom react-hot-toast
```

Library	Category	Why it earns its place
`bootstrap`	Styling	The CSS framework: grid, cards, buttons, utilities, responsive breakpoints
`react-bootstrap`	Styling + functionality	Bootstrap components as React components: `Modal`, `Navbar`, `Tabs`, `Table`, `Pagination`, `ProgressBar` — all keyboard accessible and controlled by props instead of jQuery
`react-bootstrap-icons`	Styling	Icon set that matches Bootstrap's sizing; no icon font to load
`react-router-dom`	Functionality	The required router: URLs, nested layouts, `useSearchParams`
`react-hot-toast`	Functionality	Non-blocking feedback when a book is saved (the rubric's "clear feedback" line)

> **Justify your choices in the report.** "I compared X and Y" is worth more than a list. For example: Bootstrap vs Tailwind — Bootstrap gives you accessible components out of the box (modal focus trapping, navbar collapse) which I did not want to rebuild; react-bootstrap vs raw Bootstrap JS — I wanted no jQuery-style DOM manipulation in a React app; CSS Modules vs one global stylesheet — the app has one theme, so one `index.css` plus Bootstrap utilities was less ceremony.

Check the versions landed where you expect them:

```
npm ls react react-dom react-router-dom react-bootstrap bootstrap
```

3.5 The first real config change: `base`

This one line decides whether your deployed site is a blank page.

GitHub Pages serves a project site from `https://<user>.github.io/<repo>/`, **not** from the domain root. If your built HTML asks for `/assets/index-abc.js`, the browser looks for `https://<user>.github.io/assets/index-abc.js` — which does not exist. You get a white page and a 404 in the console.

Replace `vite.config.js`:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
// Must match the GitHub repository name, exactly (case matters).
const REPO_NAME = 'shelf'
export default defineConfig({
  plugins: [react()],
  // GitHub Pages serves the site from https://<user>.github.io/<repo>/
  base: `/${REPO_NAME}/`,
```

```
})
```

3.6 `package.json` scripts and Pages deployment

Open `package.json` and make the `scripts` block look like this (keep the dependencies that are already there):

```
{
  "name": "shelf",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "homepage": "https://YOUR-USERNAME.github.io/shelf/",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "lint": "oxlint src",
    "deploy": "npm run build && gh-pages -d dist --nojekyll"
  }
}
```

- `homepage` is used by `gh-pages` and documents your live URL in one place.
- `deploy` is a single command you will run in Part 16. `--nojekyll` stops GitHub Pages from running Jekyll over the build output (see 16.2).
- `lint` gives you a fast, dependency-free sanity check while you work.

Install the deploy helper as a dev dependency:

```
npm install -D gh-pages
```

3.7 `.gitignore` and the first commit

The Vite template ships a `.gitignore`. Confirm it contains at least:

```
node_modules
dist
dist-ssr
*.local
.env
.env.*
!.env.example
.DS_Store
*.log
.vscode/*
!.vscode/extensions.json
```

Two things people forget: **never** commit `node_modules` (thousands of files, huge repo) and **never** commit `dist` if you deploy with `gh-pages` — the deployed branch holds those files, not `main`.

Now commit:

```
git add .
git commit -m "chore: scaffold Vite + React app and install bootstrap, react-router, icons, toast"
git remote add origin https://github.com/YOUR-USERNAME/shelf.git
git push -u origin main
```

Commit message style that scores well. `type: what changed` — `feat:`, `fix:`, `docs:`, `style:`, `refactor:`, `chore:`. Short summary in the imperative ("add search page", not "added"). Aim for one commit per working step, roughly 15–25 commits over the project. A history that reads like a diary of the build is exactly what "reflects the development process" means.

3.8 Run it

```
npm run dev
```

Open http://localhost:5173. You should see the Vite counter demo. If you see a blank page, open the console: it is almost always a typo in an import path.

Commit at the end of every part of this guide. It costs ten seconds and it is 6 points.

Part 4 — Bootstrap and a custom theme

4.1 Import Bootstrap once, at the top

`src/main.jsx` — start with this (routing and providers arrive in Part 5):

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import 'bootstrap/dist/css/bootstrap.min.css'
import './index.css'
import App from './App.jsx'
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Order matters: Bootstrap first, then your own `index.css`, so your overrides win.

> **StrictMode** double-invokes effects in development on purpose, to surface bugs. If your API calls appear twice in the Network tab, that is StrictMode, not a bug — in production the effect runs once.

4.2 Theme with CSS variables instead of new CSS

Bootstrap 5.3 exposes its palette as CSS custom properties, so you can restyle every component without touching a single `.btn` rule. This file is the whole visual identity of the app — change six hex values and it looks like a different product.

`src/index.css` (style section; the full file with layout rules is in Part 14):

```
:root {
  --paper: #fbf7f0;   /* page background */
  --ink: #241f1a;     /* body text */
  --muted: #6b6257;   /* secondary text */
  --accent: #b3421b;  /* links and primary buttons */
  --accent-soft: #f6e3d8;
  --line: #e6ddd0;    /* borders */
  --radius: 14px;
}
/* Feed the palette into Bootstrap's own variables. */
:root,
[data-bs-theme='light'] {
  --bs-primary: #b3421b;
  --bs-primary-rgb: 179, 66, 27;
  --bs-body-bg: #fbf7f0;
  --bs-body-color: #241f1a;
  --bs-body-font-family: 'Inter', system-ui, -apple-system, 'Segoe UI', sans-serif;
  --bs-link-color: #9a3816;
  --bs-link-hover-color: #742a10;
  --bs-border-color: #e6ddd0;
}
```

```
body {
  background: var(--paper);
  color: var(--ink);
}
.btn-primary {
  --bs-btn-bg: var(--accent);
  --bs-btn-border-color: var(--accent);
  --bs-btn-hover-bg: #963716;
  --bs-btn-hover-border-color: #963716;
  --bs-btn-active-bg: #7f2e12;
  --bs-btn-active-border-color: #7f2e12;
}
.btn-outline-primary {
  --bs-btn-color: var(--accent);
  --bs-btn-border-color: var(--accent);
  --bs-btn-hover-bg: var(--accent);
  --bs-btn-hover-border-color: var(--accent);
}
/* Visible keyboard focus everywhere, not just on buttons. */
:focus-visible {
  outline: 3px solid #1d4ed8;
  outline-offset: 2px;
}
```

Why this approach. Setting `--bs-primary` restyles buttons, links, focus rings, form controls, spinners, badges and progress bars at once. It is also the answer to "how did you make it look designed rather than default" in the report.

Contrast check before you go further. Your text on your background must reach 4.5:1 for body text. Test the pair you chose with https://webaim.org/resources/contrastchecker/. `#241f1a` on `#fbf7f0` is about 13:1 — comfortable. A slate grey on cream often is not.

4.3 Look at what you got for free

Add a throwaway `src/App.jsx` to see the theme applied, then open the page:

```
import { Button, Card, ProgressBar, Spinner } from 'react-bootstrap'
export default function App() {
  return (
    <div className="container py-5">
      <h1 className="mb-3">Theme check</h1>
      <p><Button>Primary</Button> <Button variant="outline-primary">Outline</Button></p>
      <Card className="p-3 mb-3" style={{ maxWidth: 320 }}>
        <h2 className="h5">Card</h2>
        <ProgressBar now={60} label="60%" />
      </Card>
      <Spinner animation="border" role="status" />
    </div>
  )
}
```

Resize the window: Bootstrap's grid and utilities already respond. That is the "styling library" line on the rubric handled — but it only counts if you use it, which the next parts do.

```
git add -A
git commit -m "style: theme bootstrap with a custom palette via css variables"
```

Part 5 — Routing skeleton and the shared layout

5.1 Choose the router mode before you build screens

`react-router-dom` offers two modes, and for GitHub Pages the choice matters:

Mode	URL looks like	Deep links on Pages
`BrowserRouter`	`/shelf/book/OL893414W`	**404** — Pages has no server to rewrite the path to `index.html`
`HashRouter`	`/shelf/#/book/OL893414W`	Works — the browser never sends the part after `#` to the server

The professional-looking fix is a `404.html` redirect trick. The pragmatic fix — and the one this guide uses — is clean URLs locally, hash URLs in production, so your development experience stays pleasant and your deployed site never 404s:

```
const Router = import.meta.env.PROD ? HashRouter : BrowserRouter
```

`import.meta.env.PROD` is `true` in the built bundle and `false` under `npm run dev`.

> Mention the `404.html` alternative in your report as the design decision you considered: it keeps URLs pretty, but relies on a redirect page plus sessionStorage tricks, and a broken deep link during the examiner's five-minute check costs more than a `#` in the URL.

5.2 `index.html`

Small file, three things that matter: the `<title>` (what the browser tab and Google show), the meta description, and a favicon URL that respects `base`. `%BASE_URL%` is replaced by Vite at build time, so the icon works both at `localhost:5173/` and at `youruser.github.io/shelf/`.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta
      name="description"
      content="Shelf — search millions of Open Library books, browse authors and build a reading list."
    />
    <title>Shelf — find your next book</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

A single-file SVG favicon lives in `public/favicon.svg` — no build step, no `.ico`, and it scales. Any 32×32 SVG works:

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 32 32">
  <rect width="32" height="32" rx="8" fill="#b3421b"/>
  <path d="M7 9.5c3-1.2 6-1.2 9 0v14c-3-1.2-6-1.2-9 0v-14Z" fill="#fbf7f0"/>
  <path d="M25 9.5c-3-1.2-6-1.2-9 0v14c3-1.2 6-1.2 9 0v-14Z" fill="#f6e3d8"/>
```

```
</svg>
```

5.3 `src/main.jsx` — the final version

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter, HashRouter } from 'react-router-dom'
import { Toaster } from 'react-hot-toast'
import 'bootstrap/dist/css/bootstrap.min.css'
import './index.css'
import App from './App.jsx'
import { ReadingListProvider, ShelvesProvider } from './context/AppProviders'
// On GitHub Pages there is no server to rewrite unknown paths to index.html,
// so a deep link such as /shelf/book/OL893414W would return 404. HashRouter keeps the
// route after the "#", which the browser never sends to the server. Locally we keep
// clean URLs with BrowserRouter.
const Router = import.meta.env.PROD ? HashRouter : BrowserRouter
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <Router>
      <ReadingListProvider>
        <ShelvesProvider>
          <App />
          <Toaster
            position="bottom-center"
            toastOptions={{
              style: { borderRadius: '12px', background: '#1f1b16', color: '#fffdf8' },
            }}
          />
        </ShelvesProvider>
      </ReadingListProvider>
    </Router>
  </StrictMode>,
)
```

Write this file now but keep the two providers commented out until Part 7, or create the file `src/context/AppProviders.jsx` in this part with empty stubs. Either is fine; the app will not run until both exist.

5.4 The navbar

Familiar patterns score points: a logo that goes home, three text links, a persistent search entry point, and a saved-items badge. On a phone the links collapse into a burger.

`src/components/SiteNavbar.jsx`:

```
import { useEffect, useRef } from 'react'
import { Navbar, Nav, Container, Badge } from 'react-bootstrap'
import { BookHalf, Search } from 'react-bootstrap-icons'
import { Link, NavLink, useNavigate } from 'react-router-dom'
import { useReadingListContext } from '../context/AppProviders'
/**
 * One navbar for the whole app. The only moving part is a native <dialog>
 * search box that opens with "/" or Ctrl/Cmd+K.
 */
export default function SiteNavbar() {
  const { count } = useReadingListContext()
  const dialogRef = useRef(null)
  const inputRef = useRef(null)
  const navigate = useNavigate()
  function openSearch() {
    dialogRef.current?.showModal()
    window.setTimeout(() => inputRef.current?.focus(), 0)
  }
  useEffect(() => {
    function onKeyDown(event) {
      const tag = document.activeElement?.tagName
```

```
      const isTyping = tag === 'INPUT' || tag === 'TEXTAREA' || tag === 'SELECT'
      const isShortcut = event.key === '/' || ((event.ctrlKey || event.metaKey) && event.key === 'k')
      if (isShortcut && !isTyping) {
        event.preventDefault()
        openSearch()
      }
    }
    window.addEventListener('keydown', onKeyDown)
    return () => window.removeEventListener('keydown', onKeyDown)
  }, [])
  function handleSearchSubmit(event) {
    event.preventDefault()
    const query = inputRef.current.value.trim()
    dialogRef.current?.close()
    if (query) navigate(`/explore?q=${encodeURIComponent(query)}`)
  }
  return (
    <Navbar expand="md" className="site-navbar sticky-top">
      <Container>
        <Navbar.Brand as={Link} to="/" className="d-flex align-items-center gap-2">
          <BookHalf aria-hidden="true" />
          <span>Shelf</span>
        </Navbar.Brand>
        <Navbar.Toggle aria-controls="main-nav" />
        <Navbar.Collapse id="main-nav">
          <Nav className="ms-auto align-items-md-center gap-md-1">
            <Nav.Link as={NavLink} to="/" end>Home</Nav.Link>
            <Nav.Link as={NavLink} to="/explore">Explore</Nav.Link>
            <Nav.Link as={NavLink} to="/reading-list">
              Reading list
              {count > 0 && (
                <Badge bg="primary" pill className="ms-2" aria-label={`${count} saved books`}>
                  {count}
                </Badge>
              )}
            </Nav.Link>
            <button type="button" className="btn btn-sm btn-light ms-md-2" onClick={openSearch}>
              <Search className="me-1" aria-hidden="true" />
              Search
            </button>
          </Nav>
        </Navbar.Collapse>
      </Container>
      {/* Native <dialog>: the browser handles Esc, focus trapping and the backdrop. */}
      <dialog ref={dialogRef} className="search-dialog" aria-label="Search books">
        <form className="p-4" onSubmit={handleSearchSubmit}>
          <label htmlFor="nav-search-input" className="form-label fw-semibold">
            Search books, authors, subjects
          </label>
          <input
            id="nav-search-input"
            ref={inputRef}
            type="search"
            className="form-control form-control-lg"
            placeholder="e.g. Ursula K. Le Guin"
          />
          <div className="d-flex justify-content-between align-items-center mt-3">
            <small className="text-body-secondary">Press Esc to close</small>
            <div className="d-flex gap-2">
              <button
                type="button"
                className="btn btn-outline-secondary"
                onClick={() => dialogRef.current?.close()}
              >
                Cancel
              </button>
              <button type="submit" className="btn btn-primary">Search</button>
            </div>
          </div>
        </form>
      </dialog>
    </Navbar>
  )
}
```

Three details worth understanding, because they are the kind of thing you will be asked about:

- `NavLink` from `react-router-dom` adds an `active` class to the current link automatically — that is how the navbar shows where you are. `end` on the Home link stops `/` from matching every route.
- A **native** `<dialog>` gives you Esc-to-close, a backdrop and focus trapping from the browser, with no library and no accessibility work. One caveat: jsdom (used in tests) does not implement `showModal()`, which matters in Part 15.
- `aria-hidden="true"` on decorative icons stops screen readers announcing "book half, search" before every label. Accessibility is a rubric line; it is also two words per icon.

5.5 The app shell and routes

`src/App.jsx`:

```
import { Link, Route, Routes, useLocation } from 'react-router-dom'
import { useEffect } from 'react'
import { Container } from 'react-bootstrap'
import SiteNavbar from './components/SiteNavbar'
import HomePage from './pages/HomePage'
import ExplorePage from './pages/ExplorePage'
import BookPage from './pages/BookPage'
import AuthorPage from './pages/AuthorPage'
import ReadingListPage from './pages/ReadingListPage'
export default function App() {
  const { pathname } = useLocation()
  // A real page change should start at the top, like a normal website.
  useEffect(() => {
    window.scrollTo({ top: 0, behavior: 'instant' })
  }, [pathname])
  return (
    <div className="app-shell">
      <a href="#main" className="skip-link">Skip to content</a>
      <SiteNavbar />
      <main id="main">
        <Routes>
          <Route path="/" element={<HomePage />} />
          <Route path="/explore" element={<ExplorePage />} />
          <Route path="/book/:id" element={<BookPage />} />
          <Route path="/author/:id" element={<AuthorPage />} />
          <Route path="/reading-list" element={<ReadingListPage />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </main>
      <Footer />
    </div>
  )
}
function NotFound() {
  return (
    <Container className="py-5 text-center">
      <p className="display-4 mb-2">404</p>
      <h1 className="h3">This shelf does not exist</h1>
      <p className="text-body-secondary">The page you followed is not part of the app.</p>
      <Link to="/" className="btn btn-primary">Back to home</Link>
    </Container>
  )
}
function Footer() {
  return (
    <footer className="site-footer">
      <Container className="d-flex flex-wrap justify-content-between gap-2 py-4">
        <p className="small mb-0">
          Shelf is a student project. Book data from{' '}
          <a href="https://openlibrary.org/developers/api" target="_blank" rel="noreferrer">
            Open Library
          </a>
          , an initiative of the Internet Archive.
        </p>
```

```
        <p className="small mb-0 text-body-secondary">Built with React, Vite and Bootstrap.</p>
      </Container>
    </footer>
  )
}
```

Why this structure scores well:

- **The navbar and footer are outside** `<Routes>`, so they render once and never remount. Navigation feels instant and the saved-count badge does not flicker.
- `:id` **is a route parameter**, which is what makes `/book/:id` one screen that serves every book, instead of a page per book.
- `*` **catches everything else** and gives a friendly 404 instead of a blank page — a small thing examiners notice.
- **The footer credits the API.** Required by Open Library's terms and good practice.

5.6 Screen stubs so the app runs

Create the five page files with placeholder content, so you can walk the routes before writing real features:

```
// src/pages/HomePage.jsx
export default function HomePage() {
  return <div className="container py-5"><h1>Home</h1></div>
}
```

Repeat for `ExplorePage`, `BookPage`, `AuthorPage`, `ReadingListPage` (same shape, different title).

`main.jsx` and `SiteNavbar.jsx` from above import providers and hooks that do not exist yet, so pick one of two paths — both are fine:

- **Easiest:** paste the `main.jsx` from 5.3 later. For now keep the simple version without providers, and temporarily change the navbar line `const { count } = useReadingListContext()` to `const count = 0`. Delete that placeholder the moment Part 7 gives you the real provider.
- **Neater:** write `src/context/AppProviders.jsx` and the two hooks from Part 7 now, then everything in Part 5 runs unchanged.

Whichever you choose, the point of this step is that you can visit a route and see a heading. That is your proof the router is wired correctly before any feature depends on it.

```
npm run dev
```

Visit `/#/explore`, `/book/123`, `/_nope` (404). Then commit:

```
git add -A
git commit -m "feat: add react-router routes, shared layout, navbar with search dialog"
```

Part 6 — The API layer (`fetch`, `async/await`, errors)

This is the 8-point rubric line, and it is where most projects leak marks. The examiner opens your app, throttles the network to offline, and watches what happens. If the page hangs on a spinner forever, or shows `TypeError: Failed to fetch` in the middle of the UI, you lose the "clear and user-friendly feedback" part.

6.1 The rule: components never call `fetch` directly

Put every request in one module, `src/api/openLibrary.js`. Reasons that belong in your report:

- **One place to change** if the API URL or a field name changes.
- **One place to handle errors**, so every screen fails the same friendly way.
- **Normalisation**: the API's shape is not your component's shape. `cover_i` is not a name any component should know about.
- **Testability**: you can call these functions from Node or a test without rendering anything.

6.2 The shared request wrapper

```
const BASE = 'https://openlibrary.org'
const COVERS = 'https://covers.openlibrary.org'
```

Now the wrapper. Read it slowly; every line is doing a job you will be asked about:

- `AbortController` + `setTimeout` = a request that cannot hang forever.
- `clearTimeout` in `finally` = the timer never fires after a successful response.
- `response.ok` is `false` for 404/500 without throwing, so you must check it yourself. `fetch` only rejects on network failure, never on HTTP error status.
- Status-specific messages mean the user learns something.
- `A TypeError from fetch` means offline/DNS/CORS — the message says "are you online?" because that is the only thing the user can act on.

```
const BASE = 'https://openlibrary.org'
const COVERS = 'https://covers.openlibrary.org'
/**
 * One shared fetch wrapper so every request behaves the same way:
 * it has a timeout (no infinite spinners) and it throws an Error whose
 * message is safe to show directly in the UI.
 */
async function request(url, { timeout = 12000 } = {}) {
  const controller = new AbortController()
  const timer = setTimeout(() => controller.abort(), timeout)
  try {
    const response = await fetch(url, { signal: controller.signal })
    if (!response.ok) {
      if (response.status === 404) throw new Error('We could not find that book or author.')
      if (response.status === 429) throw new Error('Too many requests. Wait a moment, then try again.')
      if (response.status >= 500) throw new Error('Open Library is having trouble. Please try again shortly.')
      throw new Error(`Request failed with status ${response.status}.`)
    }
    return await response.json()
  } catch (error) {
    if (error.name === 'AbortError') {
      throw new Error('The request took too long. Check your connection and try again.')
    }
    // fetch() rejects with a TypeError when the network itself fails
    // (offline, DNS problem, blocked request).
    if (error instanceof TypeError) {
      throw new Error('Could not reach Open Library. Are you online?')
    }
    throw error
```

```
  } finally {
    clearTimeout(timer)
  }
}
/* The fields we ask for. Asking for fewer fields keeps responses small and fast. */
const LIST_FIELDS = [
  'key',
  'title',
  'author_name',
  'author_key',
  'first_publish_year',
  'cover_i',
  'edition_count',
  'ratings_average',
  'ratings_count',
  'subject',
].join(',')
/** Turn one API document into the shape our components expect. */
export function normaliseWork(doc) {
  return {
    // "/works/OL893414W" -> "OL893414W" so we can build clean URLs
    id: doc.key.replace('/works/', ''),
    title: doc.title ?? 'Untitled',
    authors: doc.author_name ?? [],
    authorIds: (doc.author_key ?? []).map((key) => key.replace('/authors/', '')),
    year: doc.first_publish_year ?? null,
    coverId: doc.cover_i ?? null,
    editionCount: doc.edition_count ?? 0,
    rating: doc.ratings_average ? Number(doc.ratings_average.toFixed(2)) : null,
    ratingCount: doc.ratings_count ?? 0,
    subjects: (doc.subject ?? []).slice(0, 8),
  }
}
/** Bios and descriptions come back with Windows line endings and stray spaces. */
function cleanText(value) {
  return String(value ?? '')
    .replace(/\r\n?/g, '\n')
    .trim()
}
export function coverUrl(coverId, size = 'M') {
  return coverId ? `${COVERS}/b/id/${coverId}-${size}.jpg` : null
}
export function authorPhotoUrl(photoId) {
  return photoId ? `${COVERS}/a/id/${photoId}-M.jpg` : null
}
/** Free-text search across titles, authors and subjects. */
export async function searchBooks(query, { page = 1, sort = '' } = {}) {
  const params = new URLSearchParams({
    q: query,
    page: String(page),
    limit: '24',
    fields: LIST_FIELDS,
  })
  if (sort) params.set('sort', sort)
  const data = await request(`${BASE}/search.json?${params}`)
  return {
    total: data.numFound ?? 0,
    works: (data.docs ?? []).map(normaliseWork),
  }
}
/** All works by one author, best rated first. */
export async function searchByAuthor(authorId, limit = 24) {
  const params = new URLSearchParams({
    q: `author_key:${authorId}`,
    limit: String(limit),
    sort: 'rating',
    fields: LIST_FIELDS,
  })
  const data = await request(`${BASE}/search.json?${params}`)
  return { total: data.numFound ?? 0, works: (data.docs ?? []).map(normaliseWork) }
}
/**
 * A single work. Open Library splits this information over two endpoints:
 * /works/{id}.json has the title, description and subjects,
 * while the search endpoint knows the rating and first publish year.
 * We fetch both at the same time with Promise.all.
 */
export async function getBook(id) {
  const [work, meta] = await Promise.all([
    request(`${BASE}/works/${id}.json`),
```

```
    request(
      `${BASE}/search.json?${new URLSearchParams({
        q: `key:"/works/${id}"`,
        fields: 'key,first_publish_year,ratings_average,ratings_count,edition_count,subject,author_key,author_name',
        limit: '1',
      })}`,
    ),
  ])
  const extra = meta.docs?.[0] ?? {}
  return {
    id,
    title: work.title ?? 'Untitled',
    description: cleanText(
      typeof work.description === 'string' ? work.description : work.description?.value,
    ),
    subjects: work.subjects ?? extra.subject ?? [],
    coverId: work.covers?.find((cover) => cover > 0) ?? null,
    authorIds: [
      ...(work.authors ?? []).map((entry) => entry.author.key.replace('/authors/', '')),
      ...(extra.author_key ?? []).map((key) => key.replace('/authors/', '')),
    ].filter((value, index, all) => all.indexOf(value) === index),
    authors: extra.author_name ?? [],
    year: extra.first_publish_year ?? null,
    rating: extra.ratings_average ? Number(extra.ratings_average.toFixed(2)) : null,
    ratingCount: extra.ratings_count ?? 0,
    editionCount: extra.edition_count ?? work.covers?.length ?? 0,
  }
}
/** Name, bio and dates for one author. */
export async function getAuthor(id) {
  const data = await request(`${BASE}/authors/${id}.json`)
  return {
    id,
    name: data.name ?? 'Unknown author',
    bio: cleanText(typeof data.bio === 'string' ? data.bio : data.bio?.value),
    birthDate: data.birth_date ?? null,
    deathDate: data.death_date ?? null,
    photoId: data.photos?.find((photo) => photo > 0) ?? null,
    links: (data.links ?? []).slice(0, 3).map(({ title, url }) => ({ title, url })),
  }
}
/* ------------------------------------------------------------------------
 * Home screen: a few "shelves" of covers, one per subject.
 * ------------------------------------------------------------------------ */
export const SHELF_SUBJECTS = [
  { slug: 'fantasy', label: 'Fantasy' },
  { slug: 'science_fiction', label: 'Science fiction' },
  { slug: 'mystery', label: 'Mystery' },
  { slug: 'romance', label: 'Romance' },
  { slug: 'history', label: 'History' },
  { slug: 'poetry', label: 'Poetry' },
]
export async function getShelf(subject, limit = 12) {
  const params = new URLSearchParams({
    q: `subject:${subject}`,
    limit: String(limit),
    fields: 'key,title,author_name,cover_i,first_publish_year',
  })
  const data = await request(`${BASE}/search.json?${params}`)
  // Books without a cover would leave holes in the shelf, so filter them out.
  return (data.docs ?? []).map(normaliseWork).filter((work) => work.coverId)
}
/**
 * Several shelves at once. Promise.all means the user waits for the slowest
 * request instead of for the sum of all of them.
 */
export function getFeaturedShelves(count = SHELF_SUBJECTS.length) {
  return Promise.all(
    SHELF_SUBJECTS.slice(0, count).map(async ({ slug, label }) => ({
      slug,
      label,
      works: await getShelf(slug),
    })),
  )
}
```

What to notice, and what to say in the report:

1. `normaliseWork` turns `/works/OL893414W` into `OL893414W` and `cover_i` into `coverId`. Every component downstream works with `book.title`, `book.coverId`, `book.rating` — names you chose. If Open Library ever renames a field, you edit one function.
2. `??` (nullish coalescing) and `?.` (optional chaining) are doing the defensive work: `doc.author_name ?? []` means "missing authors" is an empty array, not `undefined`, so `book.authors.length` cannot crash. Watch for the third search result in the demo data — a book with no author, no cover and no rating is a normal Open Library record.
3. `cleanText` exists because real descriptions arrive with `\r\n`. Without it, splitting on `\n` produces empty paragraphs. I found this while testing against real responses, and it is a good concrete example for the reflection section of your report.
4. `getBook` calls **two** endpoints with `Promise.all`. The work endpoint has the description; the search index has the rating. `Promise.all` means the slower of the two dictates the wait — not the sum.
5. `getFeaturedShelves` fires six subject searches in parallel and labels each shelf. Sequential `for` + `await` would take six times as long.

6.3 Test the layer before you build UI on it

You do not need a test framework yet. A throwaway script in the project root, run with `node`, is enough:

```
// scratch/api-check.mjs   (delete before submitting, or move into tests/)
import { searchBooks, getBook, getAuthor, getFeaturedShelves } from '../src/api/openLibrary.js'
const search = await searchBooks('dune')
console.log('total:', search.total)
console.log('first:', search.works[0])
const book = await getBook('OL893414W')
console.log('book:', book.title, book.year, book.rating, book.authorIds)
const author = await getAuthor('OL79034A')
console.log('author:', author.name, author.photoId)
const shelves = await getFeaturedShelves(2)
console.log('shelves:', shelves.map((shelf) => `${shelf.label} (${shelf.works.length})`))
try {
  await getBook('NOT-A-REAL-ID')
} catch (error) {
  console.log('error message the user would see:', error.message)
}
```

Run it with `node scratch/api-check.mjs`. Node 20+ can import `.js` modules directly. If you see the fields you expect, your whole app is now built on solid ground. If a field is `undefined`, fix the mapping **now**, not after three screens depend on it.

```
git add -A
git commit -m "feat: add open library api layer with normalisation, timeouts and friendly errors"
```

Part 7 — One data-fetching hook (and two providers)

Every screen needs the same three things: **data**, **error**, **loading**. If you write `useState` + `useEffect` + `try/catch/finally` five times, you will eventually forget `finally` in one of them and ship a spinner that never stops. Write it once.

7.1 `useFetch`

```
import { useCallback, useEffect, useRef, useState } from 'react'
/**
 * Runs an async function and tracks its state: data, error and loading.
 * Every screen needs exactly these three things, so they all share this hook
 * instead of copy-pasting try/catch/finally blocks.
 *
 * Returns { data, error, loading, reload }.
 */
export default function useFetch(asyncFn, deps = []) {
  const [state, setState] = useState({ data: null, error: null, loading: true })
  const [nonce, setNonce] = useState(0)
  // Keep the latest function in a ref so a new inline function on every render
  // does not restart the request. The `deps` array decides when to re-run.
  const fnRef = useRef(asyncFn)
  fnRef.current = asyncFn
  useEffect(() => {
    let cancelled = false
    async function run() {
      setState({ data: null, error: null, loading: true })
      try {
        const data = await fnRef.current()
        if (!cancelled) setState({ data, error: null, loading: false })
      } catch (error) {
        if (!cancelled) setState({ data: null, error, loading: false })
      }
    }
    run()
    // Cleanup: if the user navigates away mid-request, we ignore the result
    // instead of setting state on a component that no longer exists.
    return () => {
      cancelled = true
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [...deps, nonce])
  const reload = useCallback(() => setNonce((n) => n + 1), [])
  return { ...state, reload }
}
```

Three subtleties that separate "works" from "works well":

- `fnRef` holds the latest function. Without it, an inline arrow function (`() => getBook(id)`) is a new function on every render, so a naive `useEffect(..., [fn])` would refetch forever. `deps` is the honest dependency list.
- `cancelled` prevents the classic React warning "state update on an unmounted component" when a user navigates away mid-request.
- `reload` is a counter, not a refetch function. Bumping it re-runs the effect. That is what your "Try again" buttons call.

> Why not React Query / SWR? They are excellent and I would use one on a bigger app, but they add a caching model you would then have to explain. Twenty lines you understand beat a library you do not. Say exactly this in the report — it is a design decision, which is what the reflection asks for.

7.2 `useReadingList` — state that survives a reload

This is what makes screen 5 more than a list. It is `useState` + `localStorage`, written defensively.

```
import { useCallback, useEffect, useMemo, useState } from 'react'
const STORAGE_KEY = 'shelf.reading-list.v1'
function readStored() {
  try {
    const raw = localStorage.getItem(STORAGE_KEY)
    const parsed = raw ? JSON.parse(raw) : []
    return Array.isArray(parsed) ? parsed : []
```

```
  } catch {
    // Corrupt JSON or storage disabled (private mode) should never crash the app.
    return []
  }
}
/**
 * The reading list lives in localStorage, so it survives a page reload.
 * Reading is done once on mount, writing whenever the list changes.
 */
export default function useReadingList() {
  const [items, setItems] = useState(readStored)
  useEffect(() => {
    try {
      localStorage.setItem(STORAGE_KEY, JSON.stringify(items))
    } catch {
      // Storage full or blocked: the app still works for this session.
    }
  }, [items])
  // Sync between two open tabs, a nice detail that costs four lines.
  useEffect(() => {
    function onStorage(event) {
      if (event.key === STORAGE_KEY) setItems(readStored())
    }
    window.addEventListener('storage', onStorage)
    return () => window.removeEventListener('storage', onStorage)
  }, [])
  const ids = useMemo(() => new Set(items.map((item) => item.id)), [items])
  const add = useCallback((book) => {
    setItems((current) =>
      current.some((item) => item.id === book.id) ? current : [book, ...current],
    )
  }, [])
  const remove = useCallback((id) => {
    setItems((current) => current.filter((item) => item.id !== id))
  }, [])
  const toggle = useCallback((book) => {
    setItems((current) =>
      current.some((item) => item.id === book.id)
        ? current.filter((item) => item.id !== book.id)
        : [book, ...current],
    )
  }, [])
  const clear = useCallback(() => setItems([]), [])
  return { items, ids, add, remove, toggle, clear, count: items.length }
}
```

Details that matter: reads happen **once** inside `useState(readStored)` (not in an effect, so there is no flash of empty state); every `try/catch` protects against private-browsing mode where `localStorage` throws; the `storage` event keeps two open tabs in sync; `ids` is a `Set` so `ids.has(id)` is O(1) inside a card list of 24 items.

> **Why not `useReducer`?** The actions are one-liners over an array. `useReducer` would add indirection for no gain. If you later add re-ordering, filters and notes, switch — and say so in the reflection.

7.3 Two contexts, two different jobs

The reading list must be reachable from the navbar badge, every card, the detail page and the list page. Passing it down as props through five levels is the prop-drilling problem the course talks about; a context solves it. The featured shelves are cached in a second context so Home does not refetch when you navigate back to it.

```
import { createContext, useCallback, useContext, useMemo } from 'react'
import toast from 'react-hot-toast'
import useReadingList from '../hooks/useReadingList'
import useFetch from '../hooks/useFetch'
import { getFeaturedShelves } from '../api/openLibrary'
const ReadingListContext = createContext(null)
/**
```

```
 * Makes the reading list reachable from the navbar badge, the book cards,
 * the detail page and the reading list screen without prop-drilling.
 */
export function ReadingListProvider({ children }) {
  const readingList = useReadingList()
  const toggleWithToast = useCallback(
    (book) => {
      const alreadySaved = readingList.ids.has(book.id)
      readingList.toggle(book)
      toast[alreadySaved ? 'error' : 'success'](
        alreadySaved ? `Removed “${book.title}”` : `Added “${book.title}” to your list`,
      )
    },
    [readingList],
  )
  // Small enough that a plain object is fine; no useMemo ceremony needed.
  const value = { ...readingList, toggle: toggleWithToast }
  return <ReadingListContext.Provider value={value}>{children}</ReadingListContext.Provider>
}
export function useReadingListContext() {
  const context = useContext(ReadingListContext)
  if (!context) throw new Error('useReadingListContext must be used inside <ReadingListProvider>')
  return context
}
/* ---------------------------------------------------------------------- */
const ShelvesContext = createContext(null)
/**
 * The home screen and the Explore screen both want the featured shelves.
 * Loading them once in a provider means moving between those screens is
 * instant, and the data is cached for the whole session.
 */
export function ShelvesProvider({ children }) {
  const { data, error, loading, reload } = useFetch(() => getFeaturedShelves(4), [])
  const value = useMemo(() => ({ shelves: data ?? [], error, loading, reload }), [data, error, loading, reload])
  return <ShelvesContext.Provider value={value}>{children}</ShelvesContext.Provider>
}
export function useShelves() {
  const context = useContext(ShelvesContext)
  if (!context) throw new Error('useShelves must be used inside <ShelvesProvider>')
  return context
}
```

Note two habits:

- **The guard in the hook** (`if (!context) throw`) turns "I forgot the provider" into a clear error message instead of `Cannot read properties of null`.
- `toggle` **is wrapped in** `useCallback` because it goes into every card and into the context value; without it, every card re-renders when anything in the provider changes.

> **Splitting contexts** (`ReadingListContext` and `ShelvesContext` in one file) means a shelf load does not re-render the whole tree. That is a real optimisation you can point at, rather than a claim about "using context for performance".

```
git add -A
git commit -m "feat: add useFetch and useReadingList hooks plus reading-list and shelves contexts"
```

Part 8 — Reusable UI components

Three components carry the whole app's look and its states. Build them before the screens. You will write each file once and use it twenty times.

8.1 `BookCard` and `CoverImage`

```
import { Link } from 'react-router-dom'
import { useState } from 'react'
import { Image } from 'react-bootstrap'
import { coverUrl } from '../api/openLibrary'
/**
 * A cover image with a graceful fallback. Open Library returns 404 for some
 * cover ids, and the URL alone cannot tell us, so we listen for onError.
 */
export function CoverImage({ coverId, title, size = 'M', className = '' }) {
  const [failed, setFailed] = useState(false)
  const src = coverUrl(coverId, size)
  if (!src || failed) {
    return (
      <div className={`cover-fallback ${className}`} aria-hidden="true">
        <span>{title?.slice(0, 1) ?? '?'}</span>
      </div>
    )
  }
  return (
    <Image
      src={src}
      alt={`Cover of ${title}`}
      loading="lazy"
      className={className}
      onError={() => setFailed(true)}
    />
  )
}
export function BookCard({ book, onToggleSaved, isSaved }) {
  return (
    <article className="book-card h-100">
      <Link to={`/book/${book.id}`} className="book-card__cover" aria-label={`Open ${book.title}`}>
        <CoverImage coverId={book.coverId} title={book.title} className="book-card__image" />
      </Link>
      <div className="book-card__body">
        <h3 className="book-card__title">
          <Link to={`/book/${book.id}`}>{book.title}</Link>
        </h3>
        <p className="book-card__meta text-truncate">
          {book.authors.length ? book.authors.join(', ') : 'Unknown author'}
          {book.year ? ` · ${book.year}` : ''}
        </p>
        <div className="d-flex align-items-center justify-content-between gap-2 mt-2">
          <span className="book-card__rating">
            {book.rating ? `★ ${book.rating}` : 'Not rated'}
          </span>
          <button
            type="button"
            className={`btn btn-sm ${isSaved ? 'btn-primary' : 'btn-outline-primary'}`}
            onClick={() => onToggleSaved(book)}
            aria-pressed={isSaved}
          >
            {isSaved ? 'Saved' : 'Save'}
          </button>
        </div>
      </div>
    </article>
  )
}
```

Things to notice:

- `CoverImage` **handles a real failure mode.** Open Library has cover ids that 404. An `<img>` with a broken `src` shows a torn-image icon, which looks broken. The `onError` handler swaps in a typographic fallback with the book's initial, so a missing cover looks designed.
- `loading="lazy"` defers off-screen covers. With 24 covers per page this is the difference between a snappy grid and a stutter.
- `alt` **text is meaningful**, and when there is no image the fallback is `aria-hidden` — because the title is already right below it. Duplicated announcements are an accessibility bug.
- **The Save button uses** `aria-pressed` and changes its label. A screen-reader user hears the state ("pressed"), not just a colour change. Colour alone is never enough.

8.2 The three states every data screen must have

```
import { Button, Col, Row, Spinner } from 'react-bootstrap'
import { ArrowClockwise, ExclamationTriangle, Search } from 'react-bootstrap-icons'
/** Skeleton grid shown while books are loading. */
export function BookGridSkeleton({ count = 12 }) {
  return (
    <Row xs={2} md={3} lg={4} className="g-4" aria-hidden="true">
      {Array.from({ length: count }).map((_, index) => (
        <Col key={index}>
          <div className="skeleton-card">
            <div className="skeleton skeleton--cover" />
            <div className="skeleton skeleton--line" />
            <div className="skeleton skeleton--line short" />
          </div>
        </Col>
      ))}
    </Row>
  )
}
/** Small inline spinner for buttons and toolbar areas. */
export function InlineSpinner({ label = 'Loading' }) {
  return (
    <span className="d-inline-flex align-items-center gap-2 text-body-secondary small">
      <Spinner animation="border" size="sm" role="status" />
      {label}
    </span>
  )
}
/** Friendly error panel with a retry button. Never show a raw stack trace. */
export function ErrorState({ error, onRetry, title = 'Something went wrong' }) {
  return (
    <div className="state-panel state-panel--error" role="alert">
      <ExclamationTriangle size={28} aria-hidden="true" />
      <h2 className="h5 mb-1">{title}</h2>
      <p className="mb-3">{error?.message ?? 'An unexpected error happened.'}</p>
      {onRetry && (
        <Button variant="outline-primary" onClick={onRetry}>
          <ArrowClockwise className="me-2" aria-hidden="true" />
          Try again
        </Button>
      )}
    </div>
  )
}
/** Shown when a search returns zero results. */
export function EmptyState({ title = 'No books found', message, action }) {
  return (
    <div className="state-panel">
      <Search size={28} aria-hidden="true" />
      <h2 className="h5 mb-1">{title}</h2>
      <p className="mb-3 text-body-secondary">{message}</p>
      {action}
    </div>
  )
}
```

This file is the rubric's "clear and user-friendly feedback" line made concrete:

State	Component	What the user sees
Loading	`BookGridSkeleton`	Card-shaped placeholders, so the layout does not jump when data lands
Empty	`EmptyState`	"No books matched" + a suggestion + a way out
Error	`ErrorState`	A plain-English message + a **Try again** button

Two decisions to defend in your report: a **skeleton instead of a spinner** on grids, because it communicates what is coming and prevents layout shift; and **never exposing a raw error object**, because `TypeError: Failed to fetch` is not a sentence. The API layer already converted errors into English, so `ErrorState` just prints `error.message`.

> Accessibility note: the error panel is `role="alert"`, so assistive technology announces it as soon as it appears. The skeleton is `aria-hidden` so it is not read out.

```
git add -A
git commit -m "feat: add book card, cover fallback and loading/empty/error state components"
```

Part 9 — Screen 1: Home (browse)

Purpose in one sentence: the user gets inspired and jumps somewhere, without needing to know what to type.

The Home screen is not a search box with decoration. It has three genuinely different jobs: a hero with a primary action, **editorial shelves** the user did not ask for, and a footer that credits the data source. It is the only screen that fires several requests at once.

9.1 The page

```
import { useState } from 'react'
import { Link, useNavigate } from 'react-router-dom'
import { Button, Col, Container, Form, Row } from 'react-bootstrap'
import { ArrowRight, Search } from 'react-bootstrap-icons'
import { useReadingListContext, useShelves } from '../context/AppProviders'
import { BookCard, CoverImage } from '../components/BookCard'
import { BookGridSkeleton, ErrorState } from '../components/StateViews'
import { SHELF_SUBJECTS } from '../api/openLibrary'
export default function HomePage() {
  const { shelves, error, loading, reload } = useShelves()
  const { ids, toggle } = useReadingListContext()
  const [query, setQuery] = useState('')
  const navigate = useNavigate()
  function handleSubmit(event) {
    event.preventDefault()
    const trimmed = query.trim()
    if (!trimmed) return
    navigate(`/explore?q=${encodeURIComponent(trimmed)}`)
  }
  return (
    <>
      <section className="hero">
        <Container>
          <p className="hero__eyebrow">Powered by Open Library · 40 million+ records</p>
          <h1 className="hero__title">
            Find the book you
            <br />
            did not know you wanted.
          </h1>
          <p className="hero__lead">
            Search the Open Library catalogue, follow authors, and keep a reading list that
```

```
            survives your browser restarts.
          </p>
          <Form className="hero__search" onSubmit={handleSubmit} role="search">
            <Form.Label visuallyHidden>Search for a book</Form.Label>
            <div className="hero__search-row">
              <Search className="hero__search-icon" aria-hidden="true" />
              <Form.Control
                type="search"
                size="lg"
                value={query}
                onChange={(event) => setQuery(event.target.value)}
                placeholder="Title, author or subject — try “Le Guin”"
              />
              <Button type="submit" variant="primary" size="lg">
                Search
              </Button>
            </div>
          </Form>
          <div className="hero__moods">
            <span className="text-body-secondary small me-2">In the mood for</span>
            {SHELF_SUBJECTS.map((subject) => (
              <Link
                key={subject.slug}
                to={`/explore?q=subject%3A${subject.slug}`}
                className="mood-chip"
              >
                {subject.label}
              </Link>
            ))}
          </div>
        </Container>
      </section>
      <Container className="py-5">
        <div className="d-flex justify-content-between align-items-end mb-4">
          <div>
            <h2 className="h3 mb-1">Browse the shelves</h2>
            <p className="text-body-secondary mb-0">Fresh picks from four subjects, straight from the API.</p>
          </div>
          <Link to="/explore" className="btn btn-outline-primary d-none d-sm-inline-flex">
            Explore all <ArrowRight className="ms-2" aria-hidden="true" />
          </Link>
        </div>
        {error && <ErrorState error={error} onRetry={reload} title="Could not load the shelves" />}
        {loading && <BookGridSkeleton count={4} />}
        {!loading &&
          !error &&
          shelves.map((shelf) => (
            <section key={shelf.slug} className="shelf mb-5">
              <div className="d-flex justify-content-between align-items-center mb-3">
                <h3 className="h5 mb-0">{shelf.label}</h3>
                <Link to={`/explore?q=subject%3A${shelf.slug}`} className="link-primary small">
                  See all
                </Link>
              </div>
              <div className="shelf__row">
                {shelf.works.slice(0, 6).map((book) => (
                  <Link key={book.id} to={`/book/${book.id}`} className="shelf__item">
                    <CoverImage coverId={book.coverId} title={book.title} size="M" />
                    <span className="shelf__caption">{book.title}</span>
                  </Link>
                ))}
              </div>
            </section>
          ))}
        {!loading && !error && (
          <Row className="g-4 mt-2">
            <Col lg={8}>
              <div className="callout">
                <h2 className="h5">Two ways to search</h2>
                <p className="mb-2">
                  <strong>Free text</strong> searches titles, authors and subjects at once. It is
                  what the hero search box does.
                </p>
                <p className="mb-0">
                  <strong>Subject</strong> queries like <code>subject:poetry</code> browse a whole
                  category. Use the mood chips above to try one.
```

```
                </p>
              </div>
            </Col>
            <Col lg={4}>
              <BookCard
                book={{
                  id: 'OL893414W',
                  title: 'Dune',
                  authors: ['Frank Herbert'],
                  year: 1965,
                  coverId: 11481354,
                  rating: 4.31,
                }}
                isSaved={ids.has('OL893414W')}
                onToggleSaved={toggle}
              />
            </Col>
          </Row>
        )}
      </Container>
    </>
  )
}
```

9.2 What each block is doing

The hero form. A real `<Form>` with `onSubmit`, so pressing Enter works and so the browser does not reload. The label is `visuallyHidden` — sighted users get the placeholder, screen-reader users get a name. `role="search"` on the form is a landmark, so keyboard users can jump straight to it.

The mood chips. They are `<Link>`s, not buttons, because they navigate. Each one goes to `/explore?q=subject:fantasy` — and `encodeURIComponent` is doing real work there: an unencoded `:` or space in a query string is how "my search works on my machine" happens. Reuse `SHELF_SUBJECTS` from the API module instead of a second hard-coded array; two lists drift apart within a week.

The shelves. One horizontal scroller per subject, each `grid-auto-flow: column` with `scroll-snap-type: x mandatory` (Part 14). On desktop it scrolls with the trackpad, on mobile with a thumb, and there is no carousel JavaScript to break. `slice(0, 6)` keeps the first paint light.

The 8/4 split at the bottom. `Col lg={8}` + `Col lg={4}` shows off the grid without looking like a template, and it collapses to a single column on phones.

A caveat worth understanding. The demo `BookCard` in the last row is fed a hard-coded object. It exists so the layout has something in the second column before you build the reading list. Replace it with a real component (a "Because you saved…" row) once you can, and if you leave it in, do not pretend in the report that it is API data. Honesty is graded in the AI statement, and it is noticed everywhere.

9.3 The loading path you should test right now

1. Open DevTools → Network → throttle to **Slow 3G**.
2. Reload Home. You should see shimmering card skeletons, then cover rows.
3. Throttle to **Offline**, reload. You should see the friendly error panel and a working **Try again**.
4. Go back online, press **Try again**. The shelves appear without a reload.

If step 3 shows a blank page, `ErrorState` is not wired into the `error` branch. That is the single most common lost-mark bug in this assignment.

```
git add -A
git commit -m "feat(home): hero search, subject chips and parallel featured shelves"
```

Part 10 — Screen 2: Explore (search, sort, filter, paginate)

Purpose: the user finds a specific book through a query, and can share or bookmark that exact query.

This is the most feature-dense screen and the one that will earn the most UX marks if you get one architectural idea right: **the URL is the state**.

10.1 Why the URL is the state

If you keep search, sort and page in `useState`, you lose all of this for free:

- the back button (it leaves the site instead of undoing the search);
- refreshing the page (you land on an empty form);
- sharing a result (`"look what I found"` → your friend lands on the home page);
- deep-linking from the Home hero and the author page.

`useSearchParams` makes the query string the single source of truth. Typing updates a local `draft`; **submitting** writes to the URL; the effect reacts to the URL. One consequence worth spelling out: the search does not fire on every keystroke (no debounce needed, no request storm), it fires when the user says so. Mention that trade-off in the report: "a live-search would need a 300 ms debounce; a submit-based search made the request behaviour predictable and kept the URL meaningful."

10.2 The page

```
import { useEffect, useMemo, useState } from 'react'
import { Link, useSearchParams } from 'react-router-dom'
import { Button, Col, Container, Form, Pagination, Row } from 'react-bootstrap'
import { Search, XCircle } from 'react-bootstrap-icons'
import { searchBooks } from '../api/openLibrary'
import { BookCard } from '../components/BookCard'
import { BookGridSkeleton, EmptyState, ErrorState, InlineSpinner } from '../components/StateViews'
import { useReadingListContext } from '../context/AppProviders'
const SORTS = [
  { value: '', label: 'Best match' },
  { value: 'rating', label: 'Highest rated' },
  { value: 'new', label: 'Newest' },
  { value: 'old', label: 'Oldest' },
]
const SUGGESTIONS = ['Ursula K. Le Guin', 'subject:poetry', 'artificial intelligence', 'Toni Morrison']
export default function ExplorePage() {
  // The URL is the single source of truth for the search state. That makes
  // results shareable, bookmarkable, and the back button keeps working.
  const [params, setParams] = useSearchParams()
  const query = params.get('q') ?? 'dune'
  const sort = params.get('sort') ?? ''
  const page = Math.max(Number(params.get('page')) || 1, 1)
  const [draft, setDraft] = useState(query)
  const [result, setResult] = useState({ works: [], total: 0 })
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [reloadKey, setReloadKey] = useState(0)
  const [coversOnly, setCoversOnly] = useState(false)
  const { ids, toggle } = useReadingListContext()
  // Keep the input in sync when the URL changes from outside (a chip, a link…).
  // Adjusting state during render is the React-recommended alternative to an
  // effect that would cause a second render pass.
  const [syncedQuery, setSyncedQuery] = useState(query)
  if (query !== syncedQuery) {
    setSyncedQuery(query)
    setDraft(query)
```

```
  }
  useEffect(() => {
    // This effect runs when the URL changes, not on every keystroke in the box.
    // `ignore` stops a slower, older response from overwriting a newer one.
    let ignore = false
    async function run() {
      setLoading(true)
      setError(null)
      try {
        const data = await searchBooks(query, { page, sort })
        if (!ignore) setResult(data)
      } catch (err) {
        if (!ignore) setError(err)
      } finally {
        if (!ignore) setLoading(false)
      }
    }
    run()
    return () => {
      ignore = true
    }
  }, [query, page, sort, reloadKey])
  const visibleWorks = useMemo(
    () => (coversOnly ? result.works.filter((work) => work.coverId) : result.works),
    [result.works, coversOnly],
  )
  const totalPages = Math.min(Math.ceil(result.total / 24), 50)
  function updateParams(patch) {
    const next = new URLSearchParams(params)
    Object.entries(patch).forEach(([key, value]) => {
      if (value) next.set(key, String(value))
      else next.delete(key)
    })
    setParams(next)
  }
  function handleSubmit(event) {
    event.preventDefault()
    const trimmed = draft.trim()
    if (!trimmed) return
    updateParams({ q: trimmed, page: 1 })
  }
  return (
    <Container className="py-5">
      <header className="mb-4">
        <h1 className="h2 mb-1">Explore the catalogue</h1>
        <p className="text-body-secondary mb-0">
          Type anything: a title, an author, or <code>subject:fantasy</code> to browse a category.
        </p>
      </header>
      <Form onSubmit={handleSubmit} role="search" className="explore-toolbar">
        <div className="explore-toolbar__search">
          <Search aria-hidden="true" />
          <Form.Control
            type="search"
            value={draft}
            onChange={(event) => setDraft(event.target.value)}
            placeholder="Search books…"
            aria-label="Search books"
          />
          {draft && (
            <button
              type="button"
              className="btn btn-link btn-sm text-body-secondary"
              onClick={() => setDraft('')}
              aria-label="Clear search"
            >
              <XCircle aria-hidden="true" />
            </button>
          )}
        </div>
        <Form.Select
          aria-label="Sort results"
          value={sort}
          onChange={(event) => updateParams({ sort: event.target.value, page: 1 })}
          className="explore-toolbar__sort"
```

```
        >
          {SORTS.map((option) => (
            <option key={option.value} value={option.value}>
              {option.label}
            </option>
          ))}
        </Form.Select>
        <Form.Check
          type="switch"
          id="covers-only"
          label="Covers only"
          checked={coversOnly}
          onChange={(event) => setCoversOnly(event.target.checked)}
        />
        <Button type="submit" variant="primary">
          Search
        </Button>
      </Form>
      <div className="d-flex flex-wrap justify-content-between align-items-center gap-2 my-4">
        <p className="mb-0">
          {loading ? (
            <InlineSpinner label="Searching Open Library…" />
          ) : error ? (
            <span className="text-danger">Search failed</span>
          ) : (
            <>
              <strong>{result.total.toLocaleString()}</strong> results for{' '}
              <strong>“{query}”</strong>
              {result.total > 0 && (
                <span className="text-body-secondary"> · page {page} of {totalPages}</span>
              )}
            </>
          )}
        </p>
        {!loading && !error && (
          <div className="d-flex gap-2 flex-wrap">
            {SUGGESTIONS.map((suggestion) => (
              <button
                key={suggestion}
                type="button"
                className="btn btn-sm btn-outline-secondary rounded-pill"
                onClick={() => {
                  setDraft(suggestion)
                  updateParams({ q: suggestion, page: 1 })
                }}
              >
                {suggestion}
              </button>
            ))}
          </div>
        )}
      </div>
      {error && (
        <ErrorState
          error={error}
          title="We could not run that search"
          onRetry={() => setReloadKey((key) => key + 1)}
        />
      )}
      {loading && <BookGridSkeleton count={12} />}
      {!loading && !error && visibleWorks.length === 0 && (
        <EmptyState
          title="No books matched"
          message={
            coversOnly
              ? 'Nothing with a cover here. Turn off “Covers only”, or try a different search.'
              : 'Try a shorter query, a different spelling, or browse a subject instead.'
          }
          action={
            <Link to="/explore?q=subject%3Afantasy" className="btn btn-outline-primary">
              Browse fantasy instead
            </Link>
          }
        />
      )}
```

```
      {!loading && !error && visibleWorks.length > 0 && (
        <Row xs={2} md={3} lg={4} className="g-4">
          {visibleWorks.map((book) => (
            <Col key={book.id}>
              <BookCard book={book} isSaved={ids.has(book.id)} onToggleSaved={toggle} />
            </Col>
          ))}
        </Row>
      )}
      {!loading && !error && totalPages > 1 && (
        <Pagination className="justify-content-center mt-5">
          <Pagination.Prev
            disabled={page <= 1}
            onClick={() => {
              updateParams({ page: page - 1 })
              window.scrollTo({ top: 0, behavior: 'smooth' })
            }}
          />
          <Pagination.Item active>{page}</Pagination.Item>
          <Pagination.Next
            disabled={page >= totalPages}
            onClick={() => {
              updateParams({ page: page + 1 })
              window.scrollTo({ top: 0, behavior: 'smooth' })
            }}
          />
        </Pagination>
      )}
    </Container>
  )
}
```

10.3 The five details an examiner will notice

1. `ignore` **flag.** Type two searches quickly on a slow connection: the older response can arrive after the newer one and overwrite the results with stale data. The flag discards any response from a run that is no longer current. This is called a race condition, it is real, and being able to explain it is worth more than a feature.
2. **Syncing the input during render.** When the URL changes from outside (a chip, the navbar dialog, a link), the input must show the new query. The commented pattern here (`if (query !== syncedQuery) setSyncedQuery(query)`) is React's documented alternative to calling `setState` inside an effect to mirror a prop, which causes a second render pass. If you find this confusing, an effect works too — but then remove the `syncedQuery` state and just do `useEffect(() => setDraft(query), [query])`.
3. **The total count.** `48,280 results for "dune"` (`toLocaleString`) tells the user the query worked even if their book is on page 40. Without it, an aggressive search feels like a failure.
4. **Pagination is bounded.** `Math.min(Math.ceil(total / 24), 50)` — Open Library will not page forever, and preventing "page 9,999" saves a confusing error. `updateParams` deletes parameters that are empty instead of writing `sort=`, which keeps URLs clean and comparable.
5. **The empty state earns its keep.** It is not just "nothing found"; it explains the reason when the client-side "Covers only" filter is the culprit, and offers a way out.

10.4 The toolbar is a separate visual component, not a separate file

The toolbar is a `Grid` in CSS (`Part 14`) that becomes one column on phones. On a phone the search field takes the full width, the sort select sits beside the toggle, and the Search button goes full width. No horizontal scrolling, no overflow.

Test it: DevTools → device toolbar → **iPhone SE (375 px)**. Nothing should scroll sideways.

```
git add -A
git commit -m "feat(explore): url-driven search with sort, covers filter and bounded pagination"
```

Part 11 — Screen 3: Book detail

Purpose: the user decides whether to read this specific book. It is not a bigger card; it is the screen where the two-endpoint merge, the description formatting, the author lookup and the "add to list" action live.

11.1 The page

```
import { useState } from 'react'
import { Link, useParams } from 'react-router-dom'
import { Badge, Button, Col, Container, Image, Nav, Row, Spinner, Tab } from 'react-bootstrap'
import { ArrowLeft, BookmarkHeart, BoxArrowUpRight } from 'react-bootstrap-icons'
import useFetch from '../hooks/useFetch'
import { authorPhotoUrl, getAuthor, getBook, searchByAuthor } from '../api/openLibrary'
import { CoverImage } from '../components/BookCard'
import { ErrorState } from '../components/StateViews'
import { useReadingListContext } from '../context/AppProviders'
export default function BookPage() {
  const { id } = useParams()
  const { data: book, error, loading, reload } = useFetch(() => getBook(id), [id])
  const { ids, toggle } = useReadingListContext()
  return (
    <Container className="py-4 py-lg-5">
      <Link to="/explore" className="back-link mb-4">
        <ArrowLeft aria-hidden="true" /> Back to search
      </Link>
      {loading && (
        <div className="d-flex justify-content-center py-5">
          <Spinner animation="border" role="status">
            <span className="visually-hidden">Loading book…</span>
          </Spinner>
        </div>
      )}
      {error && <ErrorState error={error} onRetry={reload} title="Could not load this book" />}
      {book && (
        <Row className="g-4 g-lg-5">
          <Col md={4} lg={3}>
            <div className="book-detail__cover">
              <CoverImage coverId={book.coverId} title={book.title} size="L" />
            </div>
            <Button
              variant={ids.has(book.id) ? 'primary' : 'outline-primary'}
              className="w-100 mt-3"
              onClick={() => toggle(book)}
              aria-pressed={ids.has(book.id)}
            >
              <BookmarkHeart className="me-2" aria-hidden="true" />
              {ids.has(book.id) ? 'In your reading list' : 'Add to reading list'}
            </Button>
            <a
              className="btn btn-link w-100 mt-1"
              href={`https://openlibrary.org/works/${book.id}`}
              target="_blank"
              rel="noreferrer"
            >
              View on Open Library <BoxArrowUpRight className="ms-1" aria-hidden="true" />
            </a>
          </Col>
          <Col md={8} lg={9}>
            <h1 className="h2 mb-2">{book.title}</h1>
            <p className="book-detail__byline">
              {book.authorIds.length > 0
                ? book.authorIds.slice(0, 3).map((authorId, index) => (
                    <span key={authorId}>
                      {index > 0 && ', '}
                      <AuthorLink authorId={authorId} fallback={book.authors[index]} />
                    </span>
                  ))
                : 'Unknown author'}
              {book.year && <span className="text-body-secondary"> · first published {book.year}</span>}
            </p>
            <div className="book-detail__stats">
              <Stat label="Rating" value={book.rating ? `★ ${book.rating}` : '—'} hint={book.ratingCount ? `${book.ratingCount} ratings` : 'No ratings yet'} />
              <Stat label="Editions" value={book.editionCount} hint="Different printings" />
              <Stat label="Subjects" value={book.subjects.length} hint="In the catalogue" />
            </div>
            <Tab.Container defaultActiveKey="about">
              <Nav variant="underline" className="mb-3">
                <Nav.Item>
                  <Nav.Link eventKey="about">About</Nav.Link>
                </Nav.Item>
                <Nav.Item>
                  <Nav.Link eventKey="authors">Authors</Nav.Link>
```

```
              </Nav.Item>
            </Nav>
            <Tab.Content>
              <Tab.Pane eventKey="about">
                {book.description ? (
                  <div className="book-detail__description">
                    {book.description.split('\n').filter(Boolean).map((paragraph, index) => (
                      <p key={index}>{paragraph}</p>
                    ))}
                  </div>
                ) : (
                  <p className="text-body-secondary">
                    Open Library has no description for this work yet. Volunteers add them
                    gradually, so this is common for older or rare books.
                  </p>
                )}
                {book.subjects.length > 0 && (
                  <div className="d-flex flex-wrap gap-2 mt-3">
                    {book.subjects.slice(0, 12).map((subject) => (
                      <Badge key={subject} bg="light" text="dark" className="subject-badge">
                        {subject}
                      </Badge>
                    ))}
                  </div>
                )}
              </Tab.Pane>
              <Tab.Pane eventKey="authors">
                {book.authorIds.length === 0 && (
                  <p className="text-body-secondary">No author is recorded for this work.</p>
                )}
                <Row className="g-3">
                  {book.authorIds.map((authorId) => (
                    <Col sm={6} key={authorId}>
                      <AuthorCard authorId={authorId} />
                    </Col>
                  ))}
                </Row>
              </Tab.Pane>
            </Tab.Content>
          </Tab.Container>
        </Col>
      </Row>
    )}
  </Container>
 )
}
/**
 * The work endpoint returns author ids but not their names, so the link loads
 * just the name it needs. Small, self-contained, and easy to cache later.
 */
function AuthorLink({ authorId, fallback }) {
  const { data: author } = useFetch(() => getAuthor(authorId), [authorId])
  return <Link to={`/author/${authorId}`}>{author?.name ?? fallback ?? 'Author'}</Link>
}
function Stat({ label, value, hint }) {
  return (
    <div className="stat">
      <span className="stat__label">{label}</span>
      <strong className="stat__value">{value}</strong>
      <span className="stat__hint">{hint}</span>
    </div>
  )
}
/**
 * Loads one author. Kept as its own component so the book page does not have
 * to wait for author data before it can render.
 */
function AuthorCard({ authorId }) {
  const { data: author, loading, error } = useFetch(() => getAuthor(authorId), [authorId])
  const { data: works } = useFetch(() => searchByAuthor(authorId, 40), [authorId])
  const [showFullBio, setShowFullBio] = useState(false)
  if (loading) {
    return (
      <div className="author-card">
        <Spinner animation="border" size="sm" role="status" />
      </div>
    )
  }
  if (error) return <ErrorState error={error} title="Author unavailable" />
  const bio = author.bio ?? ''
  const shortBio = bio.length > 260 ? `${bio.slice(0, 260)}…` : bio
  return (
    <div className="author-card h-100">
      <div className="d-flex gap-3">
        {author.photoId ? (
          <Image
            src={authorPhotoUrl(author.photoId)}
            alt={author.name}
            roundedCircle
            width={56}
            height={56}
            className="author-card__photo"
          />
        ) : (
          <div className="author-card__photo author-card__photo--empty" aria-hidden="true">
            {author.name.slice(0, 1)}
          </div>
        )}
        <div>
          <h3 className="h6 mb-1">
            <Link to={`/author/${author.id}`}>{author.name}</Link>
          </h3>
```

```
          <p className="small text-body-secondary mb-1">
            {[author.birthDate, author.deathDate].filter(Boolean).join(' – ') || 'Dates unknown'}
          </p>
          <p className="small mb-1">
            {works?.total ? `${works.total} works in Open Library` : ' '}
          </p>
          <Link to={`/author/${author.id}`} className="small">
            Open author page
          </Link>
        </div>
      </div>
      {bio && (
        <>
          <p className="small mt-3 mb-1">{showFullBio ? bio : shortBio}</p>
          {bio.length > 260 && (
            <button
              type="button"
              className="btn btn-link btn-sm p-0"
              onClick={() => setShowFullBio((value) => !value)}
            >
              {showFullBio ? 'Show less' : 'Read more'}
            </button>
          )}
        </>
      )}
    </div>
  )
}
```

11.2 The three ideas worth understanding here

`Promise.all` **inside the API layer, not in the component.** `getBook(id)` hides the fact that there are two requests. The component asks one question and gets one answer — and the detail screen cannot accidentally render "rating: loading" next to "description: loaded".

```
Component              getBook(id)                    Open Library
    |                        |                              |
    |--- getBook(id) ------>|--- GET /works/{id}.json ------>|
    |                        |<-- title, description, covers -|
    |                        |--- GET /search.json?q=key:"…" ->|
    |                        |<-- rating, year, editions -----|
    |<-- one merged object -|
```

A route parameter is a prop. `useParams()` gives you `id`; `useFetch(() => getBook(id), [id])` refetches when the route changes. Navigate from one book to a related book and the page updates without a full remount.

`AuthorCard` **loads its own data.** The author's name requires a second request (the work endpoint only gives author ids). If `BookPage` awaited it, the whole page would wait for data it does not show until a tab is clicked. A self-contained child component means each author card has its own tiny loading state. This is the pattern to describe as "separating concerns" in the report — with this exact example.

11.3 The honest empty states

Genuine Open Library records are missing descriptions and have no authors. Both are handled: the description tab explains why it is empty ("volunteers add them gradually"), the authors tab says "No author is recorded for this work." An examiner clicking a random obscure book is the most likely way your app gets tested — make that case look intentional rather than broken.

> **Tabs** (`Tab.Container` / `Tab.Pane` from react-bootstrap) are a functionality-library win: they come with `role="tab"`, `aria-selected`, and keyboard handling wired up. You could not hand-roll this correctly in ten minutes.

```
git add -A
git commit -m "feat(book): detail screen merging work and search endpoints, authors tab"
```

Part 12 — Screen 4: Author page

Purpose: the user explores one person's body of work, sorted the way they want.

Why this is a different screen from Book detail, in one sentence: the book screen answers "should I read this?", the author screen answers "what else has this person written that I might like?" — different question, different data, different actions (no save button on the author; a sort control instead).

12.1 The page

```
import { useState } from 'react'
import { Link, useParams } from 'react-router-dom'
import { Col, Container, Image, Row, Spinner } from 'react-bootstrap'
import { BoxArrowUpRight, CalendarHeart } from 'react-bootstrap-icons'
import useFetch from '../hooks/useFetch'
import { authorPhotoUrl, getAuthor, searchByAuthor } from '../api/openLibrary'
import { BookCard } from '../components/BookCard'
import { BookGridSkeleton, EmptyState, ErrorState } from '../components/StateViews'
import { useReadingListContext } from '../context/AppProviders'
export default function AuthorPage() {
  const { id } = useParams()
  const { data: author, error, loading, reload } = useFetch(() => getAuthor(id), [id])
  const { data: works, error: worksError, loading: worksLoading } = useFetch(
    () => searchByAuthor(id, 40),
    [id],
  )
  const { ids, toggle } = useReadingListContext()
  const [expanded, setExpanded] = useState(false)
  const [sortBy, setSortBy] = useState('rating')
  if (loading) {
    return (
      <Container className="py-5 text-center">
        <Spinner animation="border" role="status">
          <span className="visually-hidden">Loading author…</span>
        </Spinner>
      </Container>
    )
  }
  if (error) {
    return (
      <Container className="py-5">
        <ErrorState error={error} onRetry={reload} title="Could not load this author" />
      </Container>
    )
  }
  const bio = author.bio ?? ''
  const sortedWorks = [...(works?.works ?? [])].sort((a, b) => {
    if (sortBy === 'a-z') return a.title.localeCompare(b.title)
    if (sortBy === 'year') return (a.year ?? 9999) - (b.year ?? 9999)
    return (b.rating ?? 0) - (a.rating ?? 0)
  })
  return (
    <>
      <section className="author-hero">
        <Container>
          <Row className="align-items-center g-4">
            <Col xs="auto">
              {author.photoId ? (
                <Image
                  src={authorPhotoUrl(author.photoId)}
                  alt={author.name}
                  roundedCircle
                  className="author-hero__photo"
                />
              ) : (
                <div className="author-hero__photo author-hero__photo--empty" aria-hidden="true">
                  {author.name.slice(0, 1)}
                </div>
```

```
            )}
          </Col>
          <Col>
            <h1 className="h2 mb-1">{author.name}</h1>
            <p className="text-body-secondary mb-2">
              <CalendarHeart className="me-2" aria-hidden="true" />
              {[author.birthDate, author.deathDate].filter(Boolean).join(' – ') || 'Dates unknown'}
              {works?.total ? ` · ${works.total} works` : ''}
            </p>
            {author.links.length > 0 && (
              <div className="d-flex flex-wrap gap-3">
                {author.links.map((link) => (
                  <a key={link.url} href={link.url} target="_blank" rel="noreferrer" className="small">
                    {link.title} <BoxArrowUpRight aria-hidden="true" />
                  </a>
                ))}
              </div>
            )}
          </Col>
        </Row>
      </Container>
    </section>
    <Container className="py-5">
      {bio && (
        <div className="author-bio mb-5">
          <h2 className="h5 mb-2">About</h2>
          <p className={`mb-2 ${expanded ? '' : 'author-bio__clamped'}`}>{bio}</p>
          {bio.length > 420 && (
            <button
              type="button"
              className="btn btn-link btn-sm p-0"
              onClick={() => setExpanded((value) => !value)}
              aria-expanded={expanded}
            >
              {expanded ? 'Show less' : 'Read full biography'}
            </button>
          )}
        </div>
      )}
      <div className="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-4">
        <h2 className="h4 mb-0">Works</h2>
        <div className="btn-group btn-group-sm" role="group" aria-label="Sort works">
          {[
            { value: 'rating', label: 'Top rated' },
            { value: 'year', label: 'Oldest first' },
            { value: 'a-z', label: 'A–Z' },
          ].map((option) => (
            <button
              key={option.value}
              type="button"
              className={`btn ${sortBy === option.value ? 'btn-primary' : 'btn-outline-primary'}`}
              onClick={() => setSortBy(option.value)}
              aria-pressed={sortBy === option.value}
            >
              {option.label}
            </button>
          ))}
        </div>
      </div>
      {worksLoading && <BookGridSkeleton count={8} />}
      {worksError && <ErrorState error={worksError} title="Could not load this author's works" />}
      {!worksLoading && !worksError && sortedWorks.length === 0 && (
        <EmptyState
          title="No works recorded"
          message="Open Library does not list any works for this author yet."
          action={
            <Link to="/explore" className="btn btn-outline-primary">
              Search somewhere else
            </Link>
          }
        />
      )}
      {!worksLoading && !worksError && sortedWorks.length > 0 && (
        <Row xs={2} md={3} lg={4} className="g-4">
          {sortedWorks.map((book) => (
```

```
              <Col key={book.id}>
                <BookCard book={book} isSaved={ids.has(book.id)} onToggleSaved={toggle} />
              </Col>
            ))}
          </Row>
        )}
      </Container>
    </>
  )
}
```

12.2 Details worth pointing at

- `/authors/{id}/works.json` **is the wrong endpoint here.** It returns works with no ratings and no covers. `searchByAuthor` (`q=author_key:OL79034A&sort=rating`) returns the same books with covers, years and ratings. Choosing the endpoint that returns what the UI needs is an API-integration skill; say so in the report.
- **Client-side sorting of up to 40 items** is instant and costs nothing, versus three more API calls. Sorting a server list of thousands would not be acceptable — knowing where the line is, is the point.
- **The biography is clamped with CSS** (`-webkit-line-clamp`), with a real "Read full biography" toggle that sets `aria-expanded`. Some author bios are 3,000 words.
- `Col xs="auto"` keeps the portrait from collapsing on small screens; the name column takes the rest. It is a one-line responsive decision that looks deliberate.

```
git add -A
git commit -m "feat(author): author screen with portrait, biography and sortable works"
```

Part 13 — Screen 5: Reading list (a personal collection)

Purpose: the user keeps what they chose, and can take it elsewhere.

This screen is what proves your app is a product rather than an API viewer. It also gives you the "interactive feature" a report needs, and it works entirely offline — an examiner who opens it with the network disabled still sees a working screen.

13.1 The page

```
import { useMemo, useState } from 'react'
import { Link } from 'react-router-dom'
import { Button, Col, Container, Form, Modal, ProgressBar, Row, Table } from 'react-bootstrap'
import { BookmarkHeart, Download, Trash3 } from 'react-bootstrap-icons'
import { BookCard } from '../components/BookCard'
import { useReadingListContext } from '../context/AppProviders'
const YEAR_GOAL = 24
export default function ReadingListPage() {
  const { items, clear, remove, count } = useReadingListContext()
  const [filter, setFilter] = useState('')
  const [confirming, setConfirming] = useState(false)
  const filtered = useMemo(() => {
    const needle = filter.trim().toLowerCase()
    if (!needle) return items
    return items.filter(
      (book) =>
        book.title.toLowerCase().includes(needle) ||
        (book.authors ?? []).join(' ').toLowerCase().includes(needle),
    )
```

```
  }, [items, filter])
  const averageRating = useMemo(() => {
    const rated = items.filter((book) => book.rating)
    if (rated.length === 0) return null
    return (rated.reduce((sum, book) => sum + book.rating, 0) / rated.length).toFixed(2)
  }, [items])
  /** Export as CSV so the list is portable. Built client-side, no server needed. */
  function exportCsv() {
    const escape = (value) => `"${String(value ?? '').replaceAll('"', '""')}"`
    const rows = [
      ['Title', 'Authors', 'Year', 'Rating', 'Open Library ID'],
      ...items.map((book) => [book.title, (book.authors ?? []).join('; '), book.year, book.rating, book.id]),
    ]
    const csv = rows.map((row) => row.map(escape).join(',')).join('\n')
    const blob = new Blob([csv], { type: 'text/csv;charset=utf-8;' })
    const url = URL.createObjectURL(blob)
    const link = document.createElement('a')
    link.href = url
    link.download = 'shelf-reading-list.csv'
    link.click()
    URL.revokeObjectURL(url)
  }
  const progress = Math.min(Math.round((count / YEAR_GOAL) * 100), 100)
  return (
    <Container className="py-5">
      <header className="d-flex flex-wrap justify-content-between align-items-end gap-3 mb-4">
        <div>
          <h1 className="h2 mb-1">Your reading list</h1>
          <p className="text-body-secondary mb-0">
            {count === 0
              ? 'Nothing saved yet — the list lives in this browser.'
              : `${count} saved ${count === 1 ? 'book' : 'books'}, stored locally in this browser.`}
          </p>
        </div>
        <div className="d-flex gap-2">
          <Button variant="outline-secondary" onClick={exportCsv} disabled={count === 0}>
            <Download className="me-2" aria-hidden="true" />
            Export CSV
          </Button>
          <Button variant="outline-danger" onClick={() => setConfirming(true)} disabled={count === 0}>
            <Trash3 className="me-2" aria-hidden="true" />
            Clear
          </Button>
        </div>
      </header>
      {count > 0 && (
        <Row className="g-3 mb-4">
          <Col md={6}>
            <div className="panel p-3 h-100">
              <p className="small text-body-secondary mb-1">Yearly goal · {YEAR_GOAL} books</p>
              <ProgressBar
                now={progress}
                label={`${progress}%`}
                aria-label={`${progress} percent of your yearly goal`}
                className="mb-2"
              />
              <p className="small mb-0">
                {count >= YEAR_GOAL
                  ? 'Goal reached. Time to raise it.'
                  : `${YEAR_GOAL - count} more to reach your goal.`}
              </p>
            </div>
          </Col>
          <Col md={3}>
            <div className="panel p-3 h-100">
              <p className="small text-body-secondary mb-1">Average rating</p>
              <p className="stat__value mb-0">{averageRating ? `★ ${averageRating}` : '—'}</p>
            </div>
          </Col>
          <Col md={3}>
            <div className="panel p-3 h-100">
              <p className="small text-body-secondary mb-1">Oldest book</p>
              <p className="stat__value mb-0">
                {items.filter((book) => book.year).length
                  ? Math.min(...items.filter((book) => book.year).map((book) => book.year))
                  : '—'}
```

```
            </p>
          </div>
        </Col>
      </Row>
    )}
    {count === 0 ? (
      <div className="state-panel">
        <BookmarkHeart size={28} aria-hidden="true" />
        <h2 className="h5 mb-1">Your shelf is empty</h2>
        <p className="mb-3 text-body-secondary">
          Save a book from any card, then come back here to see it.
        </p>
        <Link to="/explore" className="btn btn-primary">
          Find something to read
        </Link>
      </div>
    ) : (
      <>
        <Form className="mb-3" onSubmit={(event) => event.preventDefault()} role="search">
          <Form.Label visuallyHidden>Filter your list</Form.Label>
          <Form.Control
            type="search"
            placeholder="Filter by title or author…"
            value={filter}
            onChange={(event) => setFilter(event.target.value)}
          />
        </Form>
        {filtered.length === 0 ? (
          <p className="text-body-secondary">No saved book matches “{filter}”.</p>
        ) : (
          <>
            {/* Table on wide screens… */}
            <div className="d-none d-lg-block">
              <Table hover responsive className="align-middle reading-table">
                <thead>
                  <tr>
                    <th scope="col">Title</th>
                    <th scope="col">Author</th>
                    <th scope="col">Year</th>
                    <th scope="col">Rating</th>
                    <th scope="col">
                      <span className="visually-hidden">Actions</span>
                    </th>
                  </tr>
                </thead>
                <tbody>
                  {filtered.map((book) => (
                    <tr key={book.id}>
                      <td>
                        <Link to={`/book/${book.id}`}>{book.title}</Link>
                      </td>
                      <td>{(book.authors ?? []).join(', ') || 'Unknown'}</td>
                      <td>{book.year ?? '—'}</td>
                      <td>{book.rating ? `★ ${book.rating}` : '—'}</td>
                      <td className="text-end">
                        <Button
                          variant="link"
                          size="sm"
                          className="text-danger"
                          onClick={() => remove(book.id)}
                          aria-label={`Remove ${book.title}`}
                        >
                          <Trash3 aria-hidden="true" />
                        </Button>
                      </td>
                    </tr>
                  ))}
                </tbody>
              </Table>
            </div>
            {/* …cards on small screens, because a table would need horizontal scrolling. */}
            <Row xs={2} sm={3} className="g-3 d-lg-none">
              {filtered.map((book) => (
                <Col key={book.id}>
                  <BookCard book={book} isSaved onToggleSaved={() => remove(book.id)} />
                </Col>
```

```
                ))}
              </Row>
            </>
          )}
        </>
      )}
      <Modal show={confirming} onHide={() => setConfirming(false)} centered>
        <Modal.Header closeButton>
          <Modal.Title className="h5">Clear your reading list?</Modal.Title>
        </Modal.Header>
        <Modal.Body>
          This removes all {count} saved books from this browser. It cannot be undone.
        </Modal.Body>
        <Modal.Footer>
          <Button variant="secondary" onClick={() => setConfirming(false)}>
            Keep them
          </Button>
          <Button
            variant="danger"
            onClick={() => {
              clear()
              setConfirming(false)
            }}
          >
            Clear list
          </Button>
        </Modal.Footer>
      </Modal>
    </Container>
  )
}
```

13.2 What it demonstrates

Element	Skill it shows
`localStorage` persistence	State management beyond `useState`, defensive coding
Responsive swap: table ↔ cards	`d-none d-lg-block` / `d-lg-none d-lg-block` — the same data, two presentations, no horizontal scroll on phones
`ProgressBar` against a 24-book goal	A derived value (`count / YEAR_GOAL`) and a library component used for a real purpose
Average rating, oldest book	`useMemo` for derived data over a list
Filter field	Local filtering, distinct from the API search — a different feature on a different screen
Export CSV	Building a `Blob`, an object URL, and a programmatic download — no server, no library
Clear with a confirmation `Modal`	Destructive actions need a confirm step; the modal traps focus for free

The CSV export is worth ten lines of your report: it is the "functionality library plus real feature" combination that reads as thought rather than tutorial-following.

```
const csv = rows.map((row) => row.map(escape).join(',')).join('\n')
const blob = new Blob([csv], { type: 'text/csv;charset=utf-8;' })
const url = URL.createObjectURL(blob)
const link = document.createElement('a')
link.href = url
link.download = 'shelf-reading-list.csv'
link.click()
URL.revokeObjectURL(url)   // do not leak the object URL
```

Note the `escape` helper: a title containing a comma or a quote breaks a naive CSV. That is a bug you can name in the reflection.

13.3 Three states of this screen

1. **Empty** — a real empty state that tells the user how to fill it (link to Explore).
2. **Populated** — cards on phones, a table on desktop.
3. **Filtered to nothing** — "No saved book matches 'xxx'." — not the same as empty, and most students conflate them.

```
git add -A
git commit -m "feat(reading-list): saved books with filter, yearly goal, csv export and confirm modal"
```

Part 14 — Styling, responsiveness and accessibility

You now have five working screens. This part makes them look like one product on a phone, a laptop and a projector. It is worth several marks and takes an evening.

14.1 The division of labour

Do not fight Bootstrap. Use it for the 80% it is good at, and write CSS only for the things it cannot know:

Use Bootstrap for	Write your own CSS for
Grid, spacing (`g-4`, `py-5`), flex helpers	The hero background, the shelf scroller, the cover fallback
Cards, buttons, badges, forms, tabs, modal, table	Book cover aspect ratio, card hover, stat blocks
Responsive display (`d-none d-lg-block`)	The explore toolbar's grid, the author hero
Dark-mode-ready variables	Anything with a name from your design ("shelf row")

14.2 The stylesheet

Add this below the theme block from Part 4. It is one file, ordered top to bottom, grouped by feature with comments — which is what makes a 600-line stylesheet readable.

```
:root {
  --paper: #fbf7f0;
  --ink: #241f1a;
  --muted: #6b6257;
  --accent: #b3421b;
  --accent-soft: #f6e3d8;
  --line: #e6ddd0;
  --radius: 14px;
}
/* Bootstrap variables, so its own components pick up our palette. */
:root,
[data-bs-theme='light'] {
  --bs-primary: #b3421b;
  --bs-primary-rgb: 179, 66, 27;
  --bs-body-bg: #fbf7f0;
  --bs-body-color: #241f1a;
  --bs-body-font-family: 'Inter', system-ui, -apple-system, 'Segoe UI', sans-serif;
  --bs-link-color: #9a3816;
  --bs-link-hover-color: #742a10;
  --bs-border-color: #e6ddd0;
}
body {
  background: var(--paper);
  color: var(--ink);
```

```
}
.btn-primary {
  --bs-btn-bg: var(--accent);
  --bs-btn-border-color: var(--accent);
  --bs-btn-hover-bg: #963716;
  --bs-btn-hover-border-color: #963716;
  --bs-btn-active-bg: #7f2e12;
  --bs-btn-active-border-color: #7f2e12;
}
.btn-outline-primary {
  --bs-btn-color: var(--accent);
  --bs-btn-border-color: var(--accent);
  --bs-btn-hover-bg: var(--accent);
  --bs-btn-hover-border-color: var(--accent);
  --bs-btn-active-bg: #963716;
  --bs-btn-active-border-color: #963716;
}
/* Visible keyboard focus everywhere, not just on buttons. */
:focus-visible {
  outline: 3px solid #1d4ed8;
  outline-offset: 2px;
}
.skip-link {
  position: absolute;
  left: 1rem;
  top: -3rem;
  z-index: 1080;
  padding: 0.5rem 1rem;
  background: var(--ink);
  color: #fff;
  border-radius: 0 0 8px 8px;
  transition: top 0.15s ease;
}
.skip-link:focus {
  top: 0;
}
/* ---------- Navbar ---------- */
.site-navbar {
  background: rgba(251, 247, 240, 0.92);
  backdrop-filter: blur(8px);
  border-bottom: 1px solid var(--line);
}
.site-navbar .navbar-brand {
  font-weight: 700;
  letter-spacing: -0.02em;
}
.site-navbar .nav-link.active {
  color: var(--accent) !important;
  font-weight: 600;
}
/* ---------- Hero ---------- */
.hero {
  padding: clamp(3rem, 8vw, 6rem) 0 clamp(2rem, 5vw, 3.5rem);
  background:
    radial-gradient(60% 60% at 15% 0%, #f7e6da 0%, transparent 70%),
    radial-gradient(50% 50% at 85% 10%, #e9eee2 0%, transparent 70%);
}
.hero__eyebrow {
  font-size: 0.8rem;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  color: var(--muted);
  margin-bottom: 1rem;
}
.hero__title {
  font-size: clamp(2.2rem, 6vw, 4rem);
  line-height: 1.05;
  letter-spacing: -0.03em;
  font-weight: 700;
  margin-bottom: 1rem;
}
.hero__lead {
  max-width: 34rem;
  color: var(--muted);
  font-size: 1.075rem;
```

```
  margin-bottom: 2rem;
}
.hero__search-row {
  position: relative;
  display: flex;
  gap: 0.5rem;
  max-width: 40rem;
}
.hero__search-icon {
  position: absolute;
  left: 1rem;
  top: 50%;
  transform: translateY(-50%);
  color: var(--muted);
  pointer-events: none;
}
.hero__search-row .form-control {
  padding-left: 2.75rem;
}
.hero__moods {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 0.5rem;
  margin-top: 1.25rem;
}
.mood-chip {
  font-size: 0.85rem;
  padding: 0.25rem 0.75rem;
  border: 1px solid var(--line);
  border-radius: 999px;
  background: #fff;
  color: var(--ink);
  text-decoration: none;
  text-transform: capitalize;
}
.mood-chip:hover {
  border-color: var(--accent);
  color: var(--accent);
}
/* ---------- Shelves ---------- */
.shelf__row {
  display: grid;
  grid-auto-flow: column;
  grid-auto-columns: clamp(110px, 22vw, 150px);
  gap: 1rem;
  overflow-x: auto;
  padding-bottom: 0.75rem;
  scroll-snap-type: x mandatory;
}
.shelf__item {
  scroll-snap-align: start;
  display: block;
  text-decoration: none;
  color: inherit;
}
.shelf__item img,
.shelf__item .cover-fallback {
  width: 100%;
  aspect-ratio: 2 / 3;
  object-fit: cover;
  border-radius: 8px;
  box-shadow: 0 6px 18px rgba(36, 31, 26, 0.14);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}
.shelf__item:hover img,
.shelf__item:focus-visible img {
  transform: translateY(-4px);
  box-shadow: 0 12px 26px rgba(36, 31, 26, 0.22);
}
.shelf__caption {
  display: block;
  font-size: 0.8rem;
  color: var(--muted);
  margin-top: 0.5rem;
```

```
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.callout {
  background: #fff;
  border: 1px solid var(--line);
  border-left: 4px solid var(--accent);
  border-radius: var(--radius);
  padding: 1.25rem 1.5rem;
  height: 100%;
}
/* ---------- Book cards ---------- */
.book-card {
  display: flex;
  flex-direction: column;
  background: #fff;
  border: 1px solid var(--line);
  border-radius: var(--radius);
  padding: 0.75rem;
  transition: box-shadow 0.2s ease, transform 0.2s ease;
}
.book-card:hover {
  box-shadow: 0 10px 28px rgba(36, 31, 26, 0.12);
  transform: translateY(-2px);
}
.book-card__cover {
  display: block;
  border-radius: 8px;
  overflow: hidden;
  background: var(--accent-soft);
}
.book-card__image,
.book-card__cover .cover-fallback {
  width: 100%;
  aspect-ratio: 2 / 3;
  object-fit: cover;
}
.book-card__body {
  padding-top: 0.75rem;
  display: flex;
  flex-direction: column;
  flex: 1;
}
.book-card__title {
  font-size: 0.95rem;
  font-weight: 600;
  line-height: 1.3;
  margin-bottom: 0.25rem;
  display: -webkit-box;
  -webkit-line-clamp: 2;
  -webkit-box-orient: vertical;
  overflow: hidden;
}
.book-card__title a {
  color: inherit;
  text-decoration: none;
}
.book-card__title a:hover {
  color: var(--accent);
}
.book-card__meta {
  font-size: 0.8rem;
  color: var(--muted);
  margin-bottom: 0;
}
.book-card__rating {
  font-size: 0.8rem;
  font-weight: 600;
  color: var(--accent);
}
.cover-fallback {
  display: grid;
  place-items: center;
  background: linear-gradient(160deg, #f2e0d3, #e8e3d6);
```

```
  color: var(--accent);
  font-size: 2rem;
  font-weight: 700;
  border-radius: 8px;
}
/* ---------- Loading + empty states ---------- */
.skeleton-card {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: var(--radius);
  padding: 0.75rem;
}
.skeleton {
  background: linear-gradient(90deg, #eee6da 25%, #f7f1e7 37%, #eee6da 63%);
  background-size: 400% 100%;
  animation: shimmer 1.4s ease-in-out infinite;
  border-radius: 6px;
}
.skeleton--cover {
  aspect-ratio: 2 / 3;
  margin-bottom: 0.75rem;
}
.skeleton--line {
  height: 0.75rem;
  margin-bottom: 0.4rem;
}
.skeleton--line.short {
  width: 55%;
}
@keyframes shimmer {
  0% {
    background-position: 100% 50%;
  }
  100% {
    background-position: 0 50%;
  }
}
.state-panel {
  text-align: center;
  padding: 3rem 1.5rem;
  border: 1px dashed var(--line);
  border-radius: var(--radius);
  background: #fff;
  color: var(--muted);
}
.state-panel--error {
  border-style: solid;
  border-color: #f0c2c2;
  background: #fff7f7;
  color: #7a2a2a;
}
/* ---------- Explore toolbar ---------- */
.explore-toolbar {
  display: grid;
  grid-template-columns: 1fr;
  gap: 0.75rem;
  align-items: center;
  background: #fff;
  border: 1px solid var(--line);
  border-radius: var(--radius);
  padding: 0.75rem;
}
.explore-toolbar__search {
  position: relative;
  display: flex;
  align-items: center;
  gap: 0.5rem;
  padding-left: 0.75rem;
  color: var(--muted);
}
.explore-toolbar__search .form-control {
  border: none;
  box-shadow: none;
}
@media (min-width: 768px) {
```

```
  .explore-toolbar {
    grid-template-columns: minmax(0, 1fr) 12rem auto auto;
  }
  .explore-toolbar__sort {
    width: 12rem;
  }
}
/* ---------- Book detail ---------- */
.back-link {
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
  font-size: 0.9rem;
  color: var(--muted);
  text-decoration: none;
}
.back-link:hover {
  color: var(--accent);
}
.book-detail__cover img,
.book-detail__cover .cover-fallback {
  width: 100%;
  border-radius: 10px;
  box-shadow: 0 16px 40px rgba(36, 31, 26, 0.2);
}
.book-detail__byline {
  font-size: 1.05rem;
  margin-bottom: 1.25rem;
}
.book-detail__stats {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 0.75rem;
  margin-bottom: 1.5rem;
}
.stat {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 10px;
  padding: 0.75rem;
}
.stat__label {
  display: block;
  font-size: 0.7rem;
  text-transform: uppercase;
  letter-spacing: 0.08em;
  color: var(--muted);
}
.stat__value {
  display: block;
  font-size: 1.25rem;
  line-height: 1.2;
}
.stat__hint {
  font-size: 0.75rem;
  color: var(--muted);
}
.book-detail__description {
  max-width: 46rem;
  line-height: 1.65;
}
.subject-badge {
  font-weight: 400;
  font-size: 0.78rem;
}
.author-card {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: var(--radius);
  padding: 1rem;
}
.author-card__photo {
  object-fit: cover;
  flex: 0 0 auto;
}
```

```
.author-card__photo--empty {
  display: grid;
  place-items: center;
  width: 56px;
  height: 56px;
  border-radius: 50%;
  background: var(--accent-soft);
  color: var(--accent);
  font-weight: 700;
}
/* ---------- Author page ---------- */
.author-hero {
  padding: 3rem 0;
  background: linear-gradient(160deg, #f7e6da 0%, #eef0e6 100%);
  border-bottom: 1px solid var(--line);
}
.author-hero__photo {
  width: clamp(96px, 18vw, 160px);
  height: clamp(96px, 18vw, 160px);
  object-fit: cover;
  border-radius: 50%;
  border: 4px solid #fff;
  box-shadow: 0 10px 30px rgba(36, 31, 26, 0.18);
}
.author-hero__photo--empty {
  display: grid;
  place-items: center;
  background: var(--accent-soft);
  color: var(--accent);
  font-size: 3rem;
  font-weight: 700;
}
.author-bio__clamped {
  display: -webkit-box;
  -webkit-line-clamp: 6;
  -webkit-box-orient: vertical;
  overflow: hidden;
}
/* ---------- Reading list ---------- */
.panel {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: var(--radius);
}
.reading-table td,
.reading-table th {
  background: transparent;
}
/* ---------- Search dialog ---------- */
.search-dialog {
  border: none;
  border-radius: var(--radius);
  padding: 0;
  width: min(32rem, calc(100vw - 2rem));
  box-shadow: 0 24px 60px rgba(36, 31, 26, 0.3);
}
.search-dialog::backdrop {
  background: rgba(36, 31, 26, 0.45);
  backdrop-filter: blur(2px);
}
/* ---------- Footer ---------- */
.site-footer {
  border-top: 1px solid var(--line);
  margin-top: 3rem;
  background: #f4eee4;
}
/* ---------- Motion preferences ---------- */
@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    animation-duration: 0.001ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.001ms !important;
    scroll-behavior: auto !important;
```

```
  }
}
/* ---------- Print ---------- */
@media print {
  .site-navbar,
  .site-footer,
  .hero__search,
  .explore-toolbar {
    display: none;
  }
}
```

14.3 Responsiveness: the four breakpoints that matter

Do not design for "phones" — design for these widths and check them in DevTools:

Width	Device	What must still work
320–375 px	iPhone SE	Navbar collapses; grid is 2 columns; toolbar stacks; no horizontal scroll
768 px	iPad portrait	Grid is 3 columns; navbar expanded; book detail is 2 columns
992 px	Small laptop	Reading list switches from cards to table
1440 px	Projector	Container is capped by Bootstrap; text lines do not run 200 characters wide

The techniques used, and the one-line reason for each:

```
/* Fluid type and spacing instead of media queries for every step. */
.hero__title { font-size: clamp(2.2rem, 6vw, 4rem); }
.hero { padding: clamp(3rem, 8vw, 6rem) 0; }
/* Covers keep the 2:3 book ratio at any width, and never squash. */
.book-card__image { width: 100%; aspect-ratio: 2 / 3; object-fit: cover; }
/* A scroller that snaps and shows partial next card, which hints "scroll me". */
.shelf__row {
  display: grid;
  grid-auto-flow: column;
  grid-auto-columns: clamp(110px, 22vw, 150px);
  gap: 1rem;
  overflow-x: auto;
  scroll-snap-type: x mandatory;
}
.shelf__item { scroll-snap-align: start; }
/* Toolbar: one column on phones, four on anything wider. */
.explore-toolbar { display: grid; grid-template-columns: 1fr; gap: 0.75rem; }
@media (min-width: 768px) {
  .explore-toolbar { grid-template-columns: minmax(0, 1fr) 12rem auto auto; }
}
```

`minmax(0, 1fr)` on the first column is the fix for the classic "search input makes the whole page wider than the viewport" bug: `1fr` alone refuses to shrink below the input's intrinsic width.

14.4 Accessibility: the checklist you can actually complete

The rubric says "at least some extent" of accessibility. These seven items are cheap and visible; tick them off in a list in your report.

1. **One `<h1>` per screen**, headings in order (`h1` → `h2` → `h3`). The card titles are `h3` because each card sits under a section `h2`.
2. **Labels on every input.** A `Form.Label` (visible or `visuallyHidden`) with a matching `htmlFor`/`id`. Placeholders are not labels; they vanish when you type.

3. **Alt text that means something**: `alt={Cover of ${title}}`, and `alt=""` or `aria-hidden` on decorative icons and on the cover fallback.
4. **Visible focus.** The global `:focus-visible { outline: 3px solid …; offset 2px }` rule means keyboard users can see where they are on every element, not just Bootstrap buttons.
5. **A skip link.** Tab once on any screen: "Skip to content" appears, and Enter jumps past the navbar to `<main id="main">`.
6. **State announced, not just coloured.** The Save button toggles its label ("Save" → "Saved") and its `aria-pressed`; the toast announces "Added Dune to your list"; the error panel is `role="alert"`.
7. **Motion respect.** Every animation is disabled when the operating system asks:

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.001ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.001ms !important;
    scroll-behavior: auto !important;
  }
}
```

14.5 Test with a keyboard, not a mouse

Put the mouse aside and do this on every screen. It takes five minutes and finds real bugs:

- **Tab** through Home: skip link → logo → Home → Explore → Reading list → Search → hero input → Search button → chips → shelf covers → cards. Focus is always visible.
- **Enter** on the navbar search button: the dialog opens, focus is in the input, **Esc** closes it.
- **Tab** inside the dialog: focus must not escape to the page behind it (native `<dialog>` handles this).
- On Explore, press Tab to the sort select: the arrow keys change the value.
- On Reading list, Tab to a remove button and press Enter: the row disappears and the count updates.

Any dead end here is a bug worth fixing rather than mentioning.

```
git add -A
git commit -m "style: responsive layout, cover aspect ratios, focus styles and reduced-motion support"
```

Part 15 — Quality assurance before you ship

15.1 The manual test matrix

Run this top to bottom and write the result in a scratch file. Fix everything. This table is also the skeleton of your report's reflection.

#	Scenario	Expected result	Seen
1	`npm run dev`, open `/`	Hero, chips, 4 shelves with covers	
2	Throttle **Slow 3G**, reload Home	Skeletons, then shelves (no layout jump)	
3	**Offline**, reload Home	Error panel + working **Try again**	
4	Back online, press Try again	Shelves load, no page reload	
5	Search "dune" on Home	Lands on Explore with results and a count	

#	Scenario	Expected result	Seen
6	Change sort to "Highest rated"	URL gains `&sort=rating`, results change	
7	Press browser **Back**	Returns to the previous sort/results, not off-site	
8	Search `subject:poetry`	Poetry results; count is large	
9	Search `qzxvbnm`	Empty state with a way out	
10	Toggle "Covers only" with a mixed result	Coverless cards disappear instantly	
11	Page 2 via Pagination	New results, page indicator updates, scroll to top	
12	Open a book	Cover, description as paragraphs, rating, editions, subjects	
13	Open an obscure book with no description	Explanatory text, not a blank panel	
14	Click an author link	Author page with portrait (or initial fallback), bio, works	
15	Sort author works A–Z	Order changes; button shows pressed state	
16	Save a book anywhere	Toast appears, navbar badge increments	
17	Reload the page	Book is still saved	
18	Open a second tab, save another book	First tab's badge updates (storage event)	
19	Reading list → Export CSV	File downloads, opens correctly in a spreadsheet	
20	Reading list → Clear → Keep them	List survives	
21	Reading list → Clear → Clear list	Empty state appears	
22	Visit `/nonsense`	404 screen with a link home	
23	Resize to 320 px on every screen	No horizontal scrollbar, everything usable	
24	Keyboard-only pass (14.5)	All controls reachable and visible	

15.2 A tiny automated safety net (optional but impressive)

You do not need tests to pass this assignment. Ten tests that run in under a second, though, are the difference between "works" and "engineered", and they give the report concrete material. This is the setup that produced the app in this guide:

```
npm install -D vitest jsdom @testing-library/react @testing-library/user-event @testing-library/jest-dom
```

```
// vitest.config.js
import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'
export default defineConfig({
  plugins: [react()],
  test: { environment: 'jsdom', globals: true, setupFiles: ['./tests/setup.js'] },
})
```

```
// tests/setup.js
import '@testing-library/jest-dom/vitest'
import { afterEach } from 'vitest'
import { cleanup } from '@testing-library/react'
// jsdom has no <dialog>; the app only calls these two methods.
if (!HTMLDialogElement.prototype.showModal) {
  HTMLDialogElement.prototype.showModal = function showModal() { this.open = true }
  HTMLDialogElement.prototype.close = function close() {
    this.open = false
    this.dispatchEvent(new Event('close'))
```

```
  }
}
afterEach(cleanup)
```

Add `"test": "vitest run"` to your scripts. The tests stub `fetch` with trimmed real responses, render the app inside a `MemoryRouter`, and assert on what the user sees:

```
it('shows a friendly error with a retry button when the network fails', async () => {
  globalThis.fetch = vi.fn(() => Promise.reject(new TypeError('Failed to fetch')))
  renderAt('/')
  await waitFor(() =>
    expect(screen.getByRole('alert')).toHaveTextContent(/could not reach open library/i),
  )
  expect(screen.getByRole('button', { name: /try again/i })).toBeInTheDocument()
})
```

Two things this found in the reference app, both of which would have shipped otherwise: descriptions containing `\r\n` rendered a phantom empty paragraph, and the same book could be saved twice if a stale list was read from another tab. Name these in the reflection — specific bugs are what "grounded in concrete examples" means.

> **Keep or cut?** If tests feel like too much before the deadline, cut them — a finished app beats an unfinished well-tested one. If you keep them, run `npm test` in front of your own eyes and mention the number of tests in the report.

15.3 The five bugs that will bite you (and their fixes)

Symptom	Cause	Fix
Blank page after deploying, console 404s on `/assets/…`	`base` is not your repo name	Part 3.5 — set `base: '/repo-name/'`
Deep link on the deployed site is a 404	`BrowserRouter` on Pages	Use `HashRouter` in production (Part 5.1)
`Objects are not valid as a React child`	Rendering an array/object from the API (`author_name`)	`.join(', ')`, or map to elements
Infinite request loop	`useEffect` dependency is a new function/object each render	The `fnRef` + explicit `deps` pattern in Part 7
"Cannot read properties of undefined" on some books	Field missing for that record	`??` defaults in `normaliseWork` (Part 6)

15.4 Lint, build, and a production preview

```
npm run lint     # oxlint: catch unused imports and hooks mistakes
npm run build    # must finish with no error
npm run preview  # serve the built bundle locally
```

`npm run preview` serves at `http://localhost:4173/shelf/` (note the `/shelf/` — that is your `base`). Open that URL, not `localhost:4173`, or you will see a 404 page that has nothing to do with your app. Walk the whole test matrix once more **against the build**, in production mode with hash URLs. This is exactly what the examiner will see.

```
git add -A
git commit -m "test: add smoke tests for routing, search states, error paths and saved books"
```

Part 16 — Deploy to GitHub Pages

16.1 The three things that must be true

1. `vite.config.js` → `base: '/<repo-name>/'` (Part 3.5).
2. A router mode that survives having no server (Part 5.1 — `HashRouter` in production).
3. A `deploy` script that publishes `dist/` to the `gh-pages` branch.

Everything else is clicking. Get these three right and deployment takes one command.

16.2 Deploy with one command

```
npm run deploy
```

That runs `vite build` and then `gh-pages -d dist --nojekyll`, which:

- creates (or updates) the `gh-pages` **branch** in your GitHub repo,
- pushes the contents of `dist/` there,
- adds a `.nojekyll` file, because `--nojekyll` asks for it. That file tells Pages to serve your files as they are instead of running them through Jekyll, which silently ignores any file or folder beginning with an underscore. Vite's own asset names do not start with _, so the site usually works without it — but it is one flag that removes a whole class of "my files disappeared after deploy" bugs, and it is also faster.

The first run will ask for GitHub credentials. Use a **personal access token** as the password (GitHub → Settings → Developer settings → Personal access tokens → Fine-grained, scope: this repository, Contents: read and write) or let the Git credential helper do it. Never paste a token into a file that ends up in git.

16.3 Turn Pages on, once

1. GitHub → your repo → **Settings** → **Pages**.
2. **Source:** Deploy from a branch.
3. **Branch:** `gh-pages` · **Folder:** `/ (root)` → **Save**.
4. Wait 1–3 minutes. The URL appears at the top: `https://YOUR-USERNAME.github.io/shelf/`.
5. Hard-reload (Ctrl+Shift+R). GitHub's CDN caches aggressively; a "blank site" five seconds after the first deploy is usually cache.

16.4 Alternative: deploy with a GitHub Action

If you prefer the repo to build itself on every push to `main` (nice for the report, and it means you cannot forget to deploy), add `.github/workflows/deploy.yml`:

```
name: Deploy to GitHub Pages
on:
  push:
    branches: [main]
  workflow_dispatch:
permissions:
  contents: read
  pages: write
  id-token: write
concurrency:
```

```
  group: pages
  cancel-in-progress: true
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-pages-artifact@v3
        with:
          path: dist
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - id: deployment
        uses: actions/deploy-pages@v4
```

Then set Pages → **Source: GitHub Actions**. Use one method or the other, not both — two publishers fighting over the same site is a confusing hour.

16.5 Verify the deployment like an examiner would

Open the live URL on **your phone**, not just your laptop, and check:

- [ ] Home loads with covers (not a blank page, no 404s in the console).
- [ ] Click through all five screens; the URL updates on every navigation.
- [ ] Refresh while on the book screen — the page still works (this is the hash-URL payoff).
- [ ] Copy the book URL into a new tab — it opens on that book.
- [ ] Save a book, refresh, it is still there.
- [ ] Turn on airplane mode, reload: the error state appears, not a broken page.
- [ ] Rotate to landscape: no layout break.

If the home page is blank, do not guess. Open DevTools → Console and Network. In order: 404 on `/assets/*` → `base`; 404 on navigation only → router mode; CORS error → wrong API; `Uncaught SyntaxError` → a stale cached bundle, hard-reload.

```
git add -A
git commit -m "ci: add github pages deploy workflow and deploy script"
```

Part 17 — The README

The rubric: "The project's README provides clear instructions for running the project." A stranger should be able to clone your repo and see the app in three commands. Write it for someone who has never seen your machine.

```
# Shelf — find your next book
A book discovery app built with React and the Open Library API. Browse curated shelves,
search 40 million catalogue records, read about authors, and keep a reading list that
survives a reload.
**Live app:** https://YOUR-USERNAME.github.io/shelf/
**API:** [Open Library](https://openlibrary.org/developers/api) (no key required)
![Home](docs/screenshot-home.png)
## Features
- **Home** — hero search, subject chips and four shelves loaded in parallel.
- **Explore** — search with sort, "covers only" filter and bounded pagination; the query
  lives in the URL, so results are shareable and the back button works.
- **Book** — description, rating, edition count and subjects, merging two API endpoints.
- **Author** — portrait, biography and a sortable list of works.
- **Reading list** — saved in `localStorage`, with filtering, a yearly goal, CSV export and
  a confirmation before clearing.
## Tech stack
| Concern | Choice |
| --- | --- |
| Build tool | Vite 8 |
| UI | React 19 |
| Routing | react-router-dom 7 |
| Styling | Bootstrap 5.3 + react-bootstrap + react-bootstrap-icons, themed with CSS variables |
| Feedback | react-hot-toast |
| Data | Open Library REST API, `fetch` + `async/await` |
## Getting started
Requires **Node.js 20.19+ or 22.12+**.
```

git clone https://github.com/YOUR-USERNAME/shelf.git cd shelf npm install npm run dev

```
Open http://localhost:5173.
```

npm run build # production build into dist/ npm run preview # serve the production build locally npm run lint # static checks npm test # component tests (if included) npm run deploy # build and publish to the gh-pages branch

```
## Project structure
```

src/ ├── api/openLibrary.js # all fetch calls, normalisation, friendly errors ├── hooks/useFetch.js # data + loading + error + retry ├── hooks/useReadingList.js # localStorage-backed saved books ├── context/AppProviders.jsx # reading list + cached shelves contexts ├── components/ # BookCard, CoverImage, SiteNavbar, StateViews ├── pages/ # Home, Explore, Book, Author, ReadingList ├── App.jsx # routes and layout └── index.css # theme variables + layout rules

```
## Deployment notes
The app is a static site on GitHub Pages, served from the `/shelf/` sub-path, so
`vite.config.js` sets `base: '/shelf/'`. GitHub Pages cannot rewrite deep links to
`index.html`, so production builds use `HashRouter`; development keeps clean URLs with
`BrowserRouter`.
## Credits
Book data and covers from [Open Library](https://openlibrary.org), an initiative of the
Internet Archive. Built as a course project.
```

Two habits that pay off: put **real screenshots** in `docs/` and reference them (a README with images reads as a finished product), and keep the **project structure** block in sync — it is the fastest way an examiner understands your architecture.

```
git add -A
```

```
git commit -m "docs: add readme with setup, structure, deployment notes and screenshots"
```

Part 18 — The report (400–600 words of your own text)

The report is 5 points and it is the easiest 5 points on offer — but only if it is honest, specific, and sectioned the way the brief asks. It is your text: if a sentence could be pasted into any other student's report unchanged, delete it.

18.1 Word budget

Section	Required	Aim for	Content
1. Project description	50–100 words	~90	What it does, who it is for, the API and libraries, both links
2. Screenshots	3–4 images	—	One per screen, captioned
3. AI usage statement	50–100 words	~90	Tools, frequency, task types, where you said no
4. Reflection	300–400 words	~350	Challenge, design decisions, what is next
Total	400–600	**~530**	

Write it in a document editor, then export to PDF named exactly `Lastname_Firstname_report.pdf`. Two A4 pages of calm layout beats four pages of cramped text.

18.2 Section 1 — project description

Answer five questions in under 100 words: what, who, which API and why, which libraries and for what, and both links. Example shape (rewrite in your own words):

> Shelf is a book discovery app for readers who know they want something but not what. It lets you browse curated subject shelves, search the Open Library catalogue, read about authors, and save books to a reading list that persists in the browser. Data comes from the Open Library REST API (`/search.json`, `/works/{id}`, `/authors/{id}`) with `fetch` and `async/await`. Bootstrap, react-bootstrap and react-bootstrap-icons handle styling and accessible components; react-router-dom handles routing; react-hot-toast confirms saves. **Repository:** github.com/you/shelf · **Live:** you.github.io/shelf/

18.3 Section 2 — screenshots

Four images, each full-window, each with a one-line caption that says **what the user is doing**, not "screenshot of the home page":

1. **Home** — "Home: hero search, subject chips and four shelves fetched in parallel."
2. **Explore** — "Explore: 48,280 results for dune with sort, covers filter and pagination state visible in the URL."
3. **Book or Author detail** — "Book detail: description and rating merged from two endpoints, with the author card loaded independently."
4. **Reading list** — "Reading list: locally saved books, yearly goal and CSV export."

How to take screenshots that do not look like a student project:

- Use a real width (1440 × 900), not a phone screenshot scaled up.
- Wait for data to load; never ship a screenshot with a skeleton or a spinner in it.
- Show a realistic state: a populated reading list, a search with results.
- Optional and effective: one extra image of the **offline error state** with the caption "Graceful failure when the network drops". It directly evidences the API-integration marks.
- Crop nothing that matters; keep the navbar visible so navigation is legible.

18.4 Section 3 — AI usage statement

The brief is specific: which tools, how often, for which tasks, concrete cases, and where you chose not to use them. All four elements, in 50–100 words. Example of the required shape:

> I used ChatGPT (GPT-5) and GitHub Copilot in VS Code, an average of two sessions per evening over nine days. I used them to explain the difference between `BrowserRouter` and `HashRouter` for GitHub Pages, to debug a `useEffect` that refetched on every render (the fix was keeping the fetch function in a ref and depending on the route parameter), to get a starting point for the CSV export with `Blob` and `createObjectURL`, and to review my error messages for tone. I did not use AI for the component structure, the screen planning, or the design/copy: I sketched those on paper first and wanted the decisions to be mine. I also wrote all final code by hand, typing it out and testing each piece against the real API rather than pasting whole files.

That is honest, specific, and it makes you look like a professional. Do not claim you wrote everything yourself if you did not — and do not understate your own work either.

18.5 Section 4 — reflection (300–400 words)

Three questions, roughly one paragraph each. The grading note says "grounded in concrete examples from the student's own code" — so every paragraph needs a filename, a function name, or a bug you fixed.

Most challenging part, and how you solved it. Pick a real story with a beginning, a symptom and a resolution. Strong examples from this build:

- The blank page after deploying. Everything worked locally; the deployed site was white with 404s on `/assets/index-….js`. Cause: GitHub Pages serves a project site from `/shelf/`, so the asset URLs needed `base: '/shelf/'` in `vite.config.js`. Then: refreshing `/shelf/book/OL893414W` returned GitHub's 404 because there is no server to rewrite the path, solved by switching to `HashRouter` for production only (`import.meta.env.PROD ? HashRouter : BrowserRouter`).
- One screen, two endpoints. The `/works/{id}.json` response has no rating, and the search endpoint has no description. The detail screen needed both, so `getBook()` now runs `Promise.all` and merges them — which also means the merging logic lives in one testable function instead of in a component.
- The request that would not stop. A naive `useEffect` with an inline `() => getBook(id)` refetched forever, because a new function is a new dependency on every render. Fixed by storing the function in `useRef` and making the route id the only dependency.

Design decisions you considered. Name the alternatives you rejected and why. This is where you prove you thought rather than copied:

- Bootstrap vs Tailwind — Bootstrap's components (modal, tabs, navbar) already handle focus management and keyboard behaviour; with Tailwind I would have rebuilt those.
- react-bootstrap vs Bootstrap's JS bundle — I did not want jQuery-style DOM manipulation inside a React tree.

- A data-fetching library (React Query/SWR) vs my own `useFetch` — the library is better at caching, but twenty lines I can explain beat a dependency I cannot.
- One context vs two — splitting the reading list from the cached shelves stops a shelf load from re-rendering every book card.
- Where the "search state" lives — URL (`useSearchParams`) not `useState`, at the cost of a submit-based search instead of live-as-you-type.
- `localStorage` instead of a backend — no accounts, no server, instant, and honest about what a static site can do.

What you would improve next. Be specific and a bit ambitious: caching API responses for a few minutes so returning to a search is instant; a real `404.html` redirect so URLs lose the `#`; reading-list notes and a "currently reading" status; virtualising the results grid for very large result sets; a dark theme using Bootstrap's `data-bs-theme`; moving normalisation into TypeScript types so a field rename fails at build time; an infinite-scroll option instead of pagination; E2E tests with Playwright against the deployed URL.

18.6 Assembly checklist before you export the PDF

- [ ] 400–600 words of your own text, counted (screenshots and captions excluded).
- [ ] All four sections present, in order, with headings that match the brief.
- [ ] Both links clickable and **tested in the PDF** (open the PDF, click them).
- [ ] 3–4 screenshots, captioned, readable at 100% zoom.
- [ ] No screenshot shows a loading skeleton, a broken cover, or `localhost` in the URL bar.
- [ ] File name `Lastname_Firstname_report.pdf`.
- [ ] Read once for typos, out loud. It takes four minutes and it is noticed.

Part 19 — The submission ZIP

The brief: "a ZIP file named Lastname_Firstname_final-project.zip including the entire project folder WITHOUT node_modules, and the report as a separate PDF."

19.1 Clean first

```
cd ..
rm -rf shelf/node_modules shelf/dist shelf/.git   # keep your local .git if you like, but it is not needed in the zip
```

Do **not** delete `.git` if you plan to keep committing after the deadline; just know that a `node_modules`-free `.git` is still several MB. Most students exclude it; the GitHub link in the report is the proof of history.

19.2 Build the zip

```
zip -r Lastname_Firstname_final-project.zip shelf \
  -x "shelf/node_modules/*" -x "shelf/dist/*" -x "*.DS_Store"
```

On Windows: right-click the folder → Send to → Compressed folder, then **verify** it does not contain `node_modules`.

19.3 Verify like an examiner

1. Unzip into a fresh folder (`~/Downloads/verify`).
2. `cd` into the project, run `npm install` then `npm run dev`, and click through all five screens. If it does not run from a clean clone, it does not pass — this step catches a missing dependency in `package.json` or a file you forgot to save.
3. Check the folder contains `README.md`, `package.json`, `vite.config.js`, `index.html`, `src/`, and `.gitignore`.
4. Check the PDF is separate: `Lastname_Firstname_report.pdf` next to the ZIP.
5. Submit **both** files, with the exact names, before the deadline (13.11.2026), not at 23:58.

Part 20 — Final rubric self-check

Print this, tick it honestly, and fix anything unticked. Anything you cannot tick is a known loss you chose — and knowing where you lose marks is part of the assignment's point.

Core structure (0–8)

- [ ] Built with Vite + React, `npm run dev` works from a clean clone.
- [ ] 3+ screens, each with a distinct purpose (browse / search / detail / person / collection).
- [ ] Routing with react-router-dom; a shared navbar; a 404 route.
- [ ] Components are reusable (`BookCard`, `CoverImage`, `StateViews`) and pages are thin.
- [ ] No giant component: nothing over ~200 lines with unrelated responsibilities.

API integration (0–8)

- [ ] A real HTTPS REST API returning JSON, clearly tied to the topic.
- [ ] All requests in `src/api/`, using `fetch` + `async/await`.
- [ ] Loading state (skeleton or spinner), not a blank area.
- [ ] Error state with a plain-English message and a working retry.
- [ ] Empty state distinct from loading and from error.
- [ ] Timeouts/`AbortController`, status-code handling, no raw error objects in the UI.
- [ ] `Promise.all` where independent calls can run in parallel.

UI and UX (0–8)

- [ ] Familiar patterns: search field with a submit action, sort select, pagination, tabs.
- [ ] Navigation is consistent and shows the current location.
- [ ] Tested at 320 / 375 / 768 / 992 / 1440 px, no horizontal scroll.
- [ ] Keyboard-only pass works; focus is always visible; skip link exists.
- [ ] Alt text, labelled inputs, one `h1` per screen, `aria-pressed` on toggles.
- [ ] An empty or half-broken record looks intentional, not broken.
- [ ] `prefers-reduced-motion` respected.

Libraries (0–5)

- [ ] A styling library is used and themed (Bootstrap + CSS variables).
- [ ] A functionality library is used for something load-bearing (router, tabs, modal, toasts).
- [ ] You can say in one sentence why each library is there instead of hand-written code.

Version control and deployment (0–6)

- [ ] Public GitHub repo, `node_modules` not committed.
- [ ] 15+ commits with meaningful messages telling the story of the build.
- [ ] README with clone → install → dev instructions, verified from a clean clone.
- [ ] Live GitHub Pages URL that loads, navigates, and survives a refresh on a deep link.
- [ ] A screenshot in the README.

Report (0–5)

- [ ] 400–600 words, four sections, correct order.
- [ ] Both links, tested in the PDF.
- [ ] 3–4 captioned screenshots of different screens.
- [ ] AI statement with tools, frequency, concrete tasks, and where you declined.
- [ ] Reflection names real files, functions and bugs from your own code.
- [ ] Named `Lastname_Firstname_report.pdf`.

If you only have two days

Do this, in this order, and stop when time runs out:

1. **Day 1 morning** — Parts 3–7: project, theme, routing, API layer, hooks. Commit five times.
2. **Day 1 afternoon** — Parts 8–10: components, Home, Explore. This alone is two screens and the API marks.
3. **Day 1 evening** — Parts 11–12: Book and Author. Three screens, three entities.
4. **Day 2 morning** — Part 13 (reading list) if time allows; otherwise polish and skip.
5. **Day 2 midday** — Part 14 and 15: responsive, keyboard, manual QA.
6. **Day 2 afternoon** — Parts 16–17: deploy, README, verify on your phone.
7. **Day 2 evening** — Parts 18–20: report, screenshots, PDF, ZIP, submit. **Submit early in the evening.** A server outage at 23:50 is not a valid excuse.

What you can drop under pressure, in order: automated tests, the author screen, CSV export, the year goal. What you must not drop: three distinct screens, the error/empty states, the README, the deployment. Those are the difference between 20/40 and 34/40.

Where to look things up

Need	Source
React fundamentals, hooks rules	https://react.dev/learn
Router APIs, `useSearchParams`, nesting	https://reactrouter.com/start/declarative/routing

Need	Source
Bootstrap classes and components	https://react-bootstrap.netlify.app/docs/components/navbar
Vite config, `base`, env variables	https://vite.dev/guide/build
GitHub Pages deployment for Vite	https://vite.dev/guide/static-deploy
Open Library API docs	https://openlibrary.org/developers/api
Which public API to use	https://github.com/public-apis/public-apis
Contrast checker	https://webaim.org/resources/contrastchecker/
Testing Library queries	https://testing-library.com/docs/queries/about

Good luck. Build the small version first, get it deployed on day one even if it is ugly, and then make it good — a live site with three screens beats a beautiful project on your laptop.